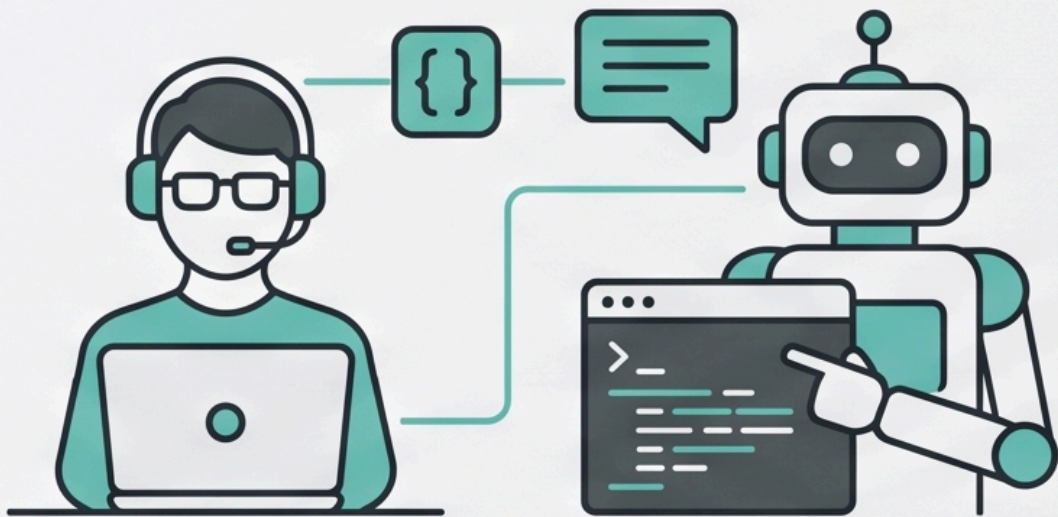


AGENTIC CODING

Basic for Junior Developers



A Practical Guide using Claude Code &
Building a Mini Ecommerce Website (Mya Yi Store)

"ကျုပ်တို့ငယ်ငယ်က ကုန်အုံတစ်ခုကို ကိုယ်တိုင်ရေးမှ ရတယ်။
အခုခေတ်ကျတော့ ကုန်ကို စကားပြောပြီး ရေးခိုင်းလို့ရနေပြီ။
ဒါကို မသိရင် ခေတ်နောက်ကျတယ်။ သိပြီး မသုံးတတ်ရင် ပိုဆိုးတယ်။"

Agentic Coding ဆိုတာ ဘာလဲဆိုတာကနေစပြီး **Claude Code** ကိုသုံးကာ **Mini Ecommerce Website** တစ်ခု (ရပ်ကွက်ထဲက ဒေါ်မြရီရဲ့ ကုန်စုံဆိုင်ကို အွန်လိုင်းတင်ပေးမယ့် "မြရီစတိုး") ကို အစအဆုံး တည်ဆောက်သွားတဲ့ လက်တွေ့သင်ခန်းစာ စာအုပ်ပါ။ Noob/Junior Developer တွေအတွက် ရည်ရွယ်ပြီး ပေါ့ပေါ့ပါးပါး စကားပြော ဟန်နဲ့ ရေးထားပါတယ်။

မှတ်ချက်

ဒီစာအုပ်ကို Junior Developer တစ်ယောက်အနေနဲ့ **Agentic Coding** ကို စတင်သင်ယူချင်သူတွေကို အဓိက ရည်ရွယ်ထားပါတယ်။ ဒီစာအုပ်ဖတ်လိုက်လို့ **Agentic Coding** ကို အပြည့်အဝ ကျွမ်းကျင်သွားမယ်လို့ မထင်ပါနဲ့။ ဘယ်ကနေစတင်ရမလဲဆိုတဲ့ bootstrap လေးလောက်ကိုပဲ ပေါ့ပေါ့ပါးပါးနဲ့ ဖြည့်ဆည်းပေးဖို့ရည်ပါတယ်။

ဘာဆောက်မလဲ

React + Vite + Tailwind CSS နဲ့ ဆောက်မယ့် "မြရီစတိုး" ရဲ့ ဝယ်ယူ ခရီးစဉ်က ဒီလို —

[ပစ္စည်းစာရင်း] → [ဈေးခြင်းတောင်း] → [Checkout Form] → [Order Summary] → [Viber ပို့]
Product List Cart (နာမည်/ဖုန်း/လိပ်စာ) (Screenshot ရိုက်) ဒေါ်မြရီဆီ ရောက်

Backend မလိုဘဲ (Server ငှားစရာ မလို) စတင်နိုင်ပြီး၊ Order ကို ဒေါ်မြရီရဲ့ Viber ဆီ Screenshot နဲ့ ပို့တဲ့ မြန်မာ့နည်း မြန်မာ့ဟန် ဖြေရှင်းချက်နဲ့ အဆုံးသတ်ပါတယ်။

မာတိကာ

1. အခန်း (၁) — Agentic Coding ဆိုတာ ဘာကောင်လဲ
2. အခန်း (၂) — Claude Code ဆိုတဲ့ လူပျိုကြီး Terminal ထဲရောက်လာခြင်း
3. အခန်း (၃) — Agent ကို အလုပ်ခိုင်းတတ်အောင် သင်ခြင်း
4. အခန်း (၄) — Mini Ecommerce Project စတင်ခြင်း (ဒေါ်မြရီ ကုန်စုံဆိုင် Online တက်မည်)
5. အခန်း (၅) — Feature တွေ တစ်ခုချင်း ဆောက်ခြင်း
6. အခန်း (၆) — Bug ရှာခြင်း၊ Test ရေးခိုင်းခြင်း၊ ဒုက္ခကနေ လွတ်မြောက်ခြင်း
7. အခန်း (၇) — Skill ဆိုတဲ့ လက်စွဲကျမ်း — တစ်ခါရေး အမြဲသုံး
8. အခန်း (၈) — Context Window နဲ့ Token စီးပွားရေး
9. အခန်း (၉) — Junior Developer တစ်ယောက်အနေနဲ့ မမေ့သင့်တဲ့ အချက်များ

အခန်း (၁) — Agentic Coding ဆိုတာ ဘာကောင်လဲ

ကျုပ်တို့ လူ့ဘောင်လောကမှာ အလုပ်ခိုင်းတတ်တဲ့သူနဲ့ အလုပ်လုပ်တတ်တဲ့သူ နှစ်မျိုးရှိတယ်။ အလုပ်လုပ်တတ်တဲ့သူက ချီးကျူးခံရတယ်။ အလုပ်ခိုင်းတတ်တဲ့သူကတော့ လခပိုများတယ်။ ဒါ လောကနိယာမ။

Agentic Coding ဆိုတာ တစ်ကြောင်းတည်း ပြောရရင် **"ကုန်ကို ကိုယ်တိုင်ရေးမနေတော့ဘဲ AI Agent တစ်ကောင်ကို ရေးခိုင်းတဲ့ အလုပ်"** ဝဲ။ ကိုယ်က ဆရာသမား၊ Agent က တပည့်။ ဒါပေမယ့် ဒီတပည့်က ထူးဆန်းတယ်။ တစ်စက္ကန့်ကို စာလုံးရေ ထောင်ချီရှိနိုင်တယ်။ ည မအိပ်ဘူး၊ လခ မတောင်းဘူး၊ ကိုယ့်ကို ဘယ်တော့မှ မထောင်ဘူး။ ဒီလောက်ကောင်းတဲ့ တပည့်မျိုး ကျုပ်တစ်သက်မှာ တစ်ခါမှ မတွေ့ဖူးဘူး။ ပြဿနာက တစ်ခုပဲ ရှိတယ်။ **ခိုင်းတတ်မှ ရတယ်။** ခိုင်းမတတ်ရင် ကိုယ်လိုချင်တာ တစ်ခု၊ သူလုပ်ပေးတာ တစ်ခု ဖြစ်ပြီး ညဘက် အိပ်မပျော်တဲ့ ရောဂါ ရတတ်တယ်။

Autocomplete ခေတ်ကနေ Agent ခေတ်ဆီ

နည်းနည်း သမိုင်းကြောင်း ပြန်ကောက်ရအောင်။

ပထမခေတ်က **Autocomplete ခေတ်**။ ကိုယ်က `for` လို့ ရိုက်လိုက်ရင် သူက `for (let i = 0; ...)` ဆိုပြီး ဖြည့်ပေးတယ်။ ဒါက ထမင်းစားနေရင်း ဘေးကလူက ဟင်းခပ်ပေးတာမျိုး။ ကျေးဇူးတင်စရာပေမယ့် ထမင်းကတော့ ကိုယ်ပဲ စားရတယ်။

ဒုတိယခေတ်က **Chat ခေတ်**။ Browser ထဲမှာ AI ကို "login form တစ်ခု ရေးပေးပါ" လို့ တောင်း၊ သူပြန်ပေးတဲ့ ကုန်ကို Copy ကူး၊ ကိုယ့် Editor ထဲ Paste ထည့်၊ Error တက်၊ Error ကို ပြန် Copy ကူး၊ AI ဆီ ပြန်သွားမေး။ ဒီလိုနဲ့ တစ်နေကုန် Copy-Paste သမားကြီး ဖြစ်နေတယ်။ လက်ညှိုးက ကုန်ရေးလို့ ညောင်းတာ မဟုတ်ဘဲ Ctrl+C, Ctrl+V နှိပ်လို့ ညောင်းတဲ့ခေတ်။

တတိယခေတ်ကတော့ အခု ကျုပ်တို့ပြောမယ့် **Agent ခေတ်**။ ဒီခေတ်မှာ AI က Browser ထဲမှာ ထိုင်မနေတော့ဘူး။ **ကိုယ့်ကွန်ပျူတာထဲ၊ ကိုယ့် Project Folder ထဲ ကိုယ်တိုင် ဆင်းလာတယ်။** ဖိုင်တွေကို သူ့ဘာသာသူ ဖွင့်ဖတ်တယ်။ ကုန်ကို သူ့ဘာသာသူ ရေးတယ်။ ရေးပြီးရင် Run ကြည့်တယ်။ Error တက်ရင် သူ့ဘာသာသူ ပြန်ပြင်တယ်။ ပြင်ပြီးရင် ထပ် Run တယ်။ အောင်မြင်တဲ့အထိ ဒီလို ထပ်ခါတလဲလဲ လုပ်တယ်။

ဥပမာ ပြောရရင် —

- **Autocomplete** ဆိုတာ ဟင်းချက်နေတုန်း ဘေးကနေ ငရုတ်သီး လှမ်းပေးတဲ့သူ။
- **Chat AI** ဆိုတာ ဟင်းချက်နည်း ဖုန်းထဲကနေ ရွတ်ပြပေးတဲ့သူ။ ချက်ရတာက ကိုယ်။
- **Agent** ဆိုတာ မီးဖိုချောင်ထဲ ကိုယ်တိုင်ဝင်၊ ဟင်းချက်၊ မြည်းကြည့်၊ ငန်ရင် ရေထပ်ထည့်၊ ပြီးမှ "ဆရာ ဟင်းကျက်ပါပြီ" လို့ လာသတင်းပို့တဲ့ ထမင်းချက်ဆရာ။

ကိုယ့်တာဝန်က ဘာလဲ။ **ဘာဟင်းစားချင်လဲ ရှင်းရှင်းလင်းလင်း ပြောဖို့နဲ့၊ ဟင်းကျက်လာရင် စားလို့ရမရ မြည်းကြည့်ဖို့ပဲ။** ဒီနှစ်ခုတော့ ဘယ်သူမှ ကိုယ့်ကိုယ်စား လုပ်မပေးနိုင်ဘူး။

Agent ဆိုတာ တကယ်ရော ဘာလုပ်နိုင်လို့လဲ

"Agent" ဆိုတဲ့ စကားလုံးက ခေတ်စားလွန်းလို့ ဘာမဆို Agent လို့ ခေါ်နေကြတယ်။ တကယ့် သဘောတရားက ရှိရှိလေးပါ။ Agent ဆိုတာ ရည်မှန်းချက်တစ်ခုအတွက် ကိုယ့်ဘာသာကိုယ် ဆုံးဖြတ်ပြီး အဆင့်ဆင့် လုပ်ဆောင် နိုင်တဲ့ AI ကို ခေါ်တာ။

သူ့မှာ လက်နက် သုံးမျိုး ရှိတယ် —

1. **ဖိုင်ဖတ်နိုင်တယ်၊ ရေးနိုင်တယ်** — ကိုယ့် Project ထဲက ကုဒ်ဖိုင်တွေကို ဖွင့်ကြည့်ပြီး ပြင်နိုင်တယ်၊ အသစ် ဆောက်နိုင်တယ်။
2. **Command တွေ Run နိုင်တယ်** — Terminal ထဲမှာ `npm install` တို့၊ `npm test` တို့ကို သူ့ဘာသာသူ ရိုက်နိုင်တယ်။
3. **ရလဒ်ကြည့်ပြီး ဆက်စဉ်းစားနိုင်တယ်** — Test Fail ရင် Error ကိုဖတ်၊ ဘာလို့ Fail လဲ စဉ်းစား၊ ပြင်၊ ပြန် စမ်း။ ဒီသံသရာကို အောင်မြင်တဲ့အထိ လည်တယ်။

ဒီသုံးချက် ပေါင်းလိုက်တော့ "login page လေး ဆောက်ပေးပါ" လို့ တစ်ကြောင်းပြောရုံနဲ့ ဖိုင် ငါးဖိုင်၊ ခြောက်ဖိုင် ဖန်တီးပြီး၊ Test ပါရေးပြီး၊ Run ပြုပြီးမှ ရပ်တဲ့ အလုပ်သမားတစ်ယောက် ရလာတယ်။

ဒါဆို Developer တွေ အလုပ်ပြုတ်တော့မှာလား

ဒီမေးခွန်းကို လမ်းထိပ်က လက်ဖက်ရည်ဆိုင်မှာရော၊ Facebook ပေါ်မှာရော တစ်နေ့ အနည်းဆုံး သုံးခါလောက် ကြားရတယ်။

အဖြေက — **ကုဒ်ရိုက်တဲ့အလုပ်ကတော့ ပြုတ်မယ်။ ဒါပေမယ့် Developer ဆိုတာ ကုဒ်ရိုက်တဲ့သူ မဟုတ်ဘူး။** ဒါ ကွဲကွဲပြားပြား သိထားဖို့ လိုတယ်။

ဆောက်လုပ်ရေး အင်ဂျင်နီယာတစ်ယောက်ကို ကြည့်ပါ။ သူက အုတ်စီတာ မဟုတ်ဘူး။ အုတ်စီတာက လက်သမား၊ ပန်းရန်ဆရာတွေ အလုပ်။ အင်ဂျင်နီယာအလုပ်က **ဘာဆောက်မလဲ ဆုံးဖြတ်တာ၊ ပုံစံချတာ၊ ဆောက်နေတာ မှန်မမှန် စစ်ဆေးတာ၊ တိုက်ပြိုမယ့် အန္တရာယ် ကြိုမြင်တာ။** Agent ခေတ်ရဲ့ Developer ဆိုတာ လည်း ဒီသဘောပါပဲ။ Agent က အုတ်စီပေးမယ်၊ ကိုယ်က အင်ဂျင်နီယာ လုပ်ရမယ်။

ဒါကြောင့် Junior Developer တွေအတွက် သတင်းကောင်းရော သတင်းဆိုးရော ရှိတယ်။

သတင်းကောင်းက — အရင်ကဆို Senior တစ်ယောက် ဖြစ်ဖို့ ကုဒ်ကြောင်းရေ သိန်းချီ ရေးဖူးမှ ရတယ်။ အခု တော့ Agent ကို ကောင်းကောင်း ခိုင်းတတ်ရင် တစ်ယောက်တည်းနဲ့ Project တစ်ခုလုံး ပြီးအောင် လုပ်နိုင်တဲ့ ခေတ် ရောက်နေပြီ။

သတင်းဆိုးက — Agent ရေးပေးတဲ့ကုဒ်ကို **နားမလည်ဘဲ ယုံပြီးသုံးတဲ့ Developer** ဟာ ကားမောင်းနည်း ကောင်းကောင်းမတတ်ဘဲ အဝေးပြေးလမ်းပေါ် တက်မောင်းတဲ့သူနဲ့ တူတယ်။ လမ်းဖြောင့်နေသရွေ့တော့ ဘာမှမ ဖြစ်ဘူး၊ ကိုယ်က ကားမောင်းတော်တယ်လို့တောင် ထင်ရသေးတယ်။ ဒါပေမယ့် လမ်းကွေ့တစ်ခု ရုတ်တရက်

ရောက်လာတဲ့အခါ၊ ဟိုဘက်က ကားတစ်စီး ဖြတ်ထွက်လာတဲ့အခါကျမှ ကားကို ကိုယ်ကိုယ်တိုင် မထိန်းတတ်မှန်း သိရတော့တယ်။ အဲဒီအချိန်ကျ နောင်တရဖို့ မမီတော့ဘူး။

ဒီစာအုပ်ရဲ့ ကတိ

ဒီစာအုပ်ထဲမှာ ကျုပ်တို့ **Claude Code** ဆိုတဲ့ Agent တစ်ကောင်ကို သုံးပြီး၊ **Mini Ecommerce Website** တစ်ခုကို အစအဆုံး ဆောက်ကြမယ်။ ဆိုင်နာမည်ကတော့ ကျုပ်တို့ရုပ်ကွက်ထဲက ဒေါ်မြရီရဲ့ ကုန်စုံဆိုင်ကို အွန်လိုင်းတင်ပေးမယ့် "မြရီစတိုး"။

ဖတ်ပြီးရင် ကိုယ်တိုင် လိုက်လုပ်ကြည့်ပါ။ Agentic Coding ဆိုတာ စာဖတ်ပြီး တတ်တဲ့ ပညာ မဟုတ်ဘူး။ **ခိုင်းကြည့်၊ မှားကြည့်၊ ပြင်ကြည့်မှ တတ်တဲ့ ပညာ။** စက်ဘီးစီးသင်တာနဲ့ အတူတူပဲ။ စက်ဘီးစီးနည်း စာအုပ် အုပ်တစ်ရာ ဖတ်ဖတ်၊ မစီးဖူးရင် လဲမှာပဲ။

လက်တွေ့လုပ်ကြည့်ရန်

Claude Code မတင်ရသေးလည်း ရပါတယ်။ အခုလောလောဆယ် ကိုယ်လုပ်ချင်နေတဲ့ Project (သို့) Feature တစ်ခုကို ရွေးပြီး "ဒါကို Agent တစ်ကောင်ကို ခိုင်းရမယ်ဆိုရင် ဘယ်လို အဆင့်ဆင့် ခွဲပြောရမလဲ" ဆိုတာ စာရွက်ပေါ်မှာ ၃-၅ ဆင့် ချရေးကြည့်ပါ။ ဒီအလေ့အကျင့်က စာအုပ်တစ်လျှောက်လုံး အသုံးဝင်ပါလိမ့်မယ်။

ကဲ — နောက်အခန်းမှာ Claude Code ဆိုတဲ့ ကောင်လေးကို ကိုယ့်စက်ထဲ ခေါ်သွင်းကြရအောင်။

 မာတိကာ · အခန်း (၂) →

အခန်း (၂) — Claude Code ဆိုတဲ့ လူပျိုကြီး Terminal ထဲရောက်လာခြင်း

Claude Code ဆိုတာ ဘာလဲ

Claude Code ဆိုတာ Anthropic ကုမ္ပဏီက ထုတ်တဲ့ **Terminal ထဲမှာ အလုပ်လုပ်တဲ့ AI Coding Agent** တစ်ခုပါ။ Browser ထဲက Chat AI တွေလို မဟုတ်ဘူး။ သူက ကိုယ့်ကွန်ပျူတာထဲမှာ တကယ်နေတယ်။ ကိုယ့် Project Folder ထဲ ဝင်ထိုင်ပြီး ဖိုင်ဖတ်တယ်၊ ကုဒ်ရေးတယ်၊ Command တွေ Run တယ်။

Terminal ဆိုတဲ့ စကားကြားရုံနဲ့ "ဟာ... အနက်ရောင် Screen ကြီး၊ စာတန်းတွေ တရစပ် ပြေးနေတာကြီး၊ ဟက်ကားတွေ သုံးတဲ့ဟာကြီး" ဆိုပြီး လန့်မသွားပါနဲ့။ Terminal ဆိုတာ ကွန်ပျူတာနဲ့ စာရိုက်ပြီး စကားပြောတဲ့ နေရာလေးပါပဲ။ Messenger နဲ့ သဘောချင်း အတူတူပဲ။ ကွာတာက တစ်ဖက်က ပြန်တဲ့ကောင်က လူမဟုတ်ဘဲ ကွန်ပျူတာ ဖြစ်နေတာပဲ ရှိတယ်။

ကြိုတင် ပြင်ဆင်စရာ

Claude Code သုံးဖို့ လိုအပ်တာ နှစ်ခုပဲ —

1. **ကွန်ပျူတာ** — macOS၊ Windows (10 နောက်ပိုင်း)၊ Linux — ဘာမဆိုရတယ်။
2. **Claude Account** — Claude Code က အခမဲ့ Plan နဲ့ မရဘူး။ Pro၊ Maxi၊ Team၊ Enterprise Plan တစ်ခုခု ဒါမှမဟုတ် API သုံးလို့ရတဲ့ Console Account လိုတယ်။ ဒါက ကုန်ကျစရိတ် ရှိတယ်ဆိုတာ ကြိုသိထားပါ။ "အလကားရတာမှ အကောင်းစား" ဆိုတဲ့ စကားက ဈေးထဲမှာပဲ မှန်တာ။ AI လောကမှာတော့ GPU ဆိုတဲ့ကောင်က လျှပ်စစ်မီးကို ထမင်းလို စားတာမို့ အလကား ကျွေးလို့ မရဘူး။

Install လုပ်ခြင်း

နည်းလမ်း အမျိုးမျိုးရှိပေမယ့် အလွယ်ဆုံးက Anthropic ရဲ့ Native Installer ပဲ။

သတိ — အောက်မှာ ဖော်ပြထားတဲ့ install command တွေနဲ့ Plan အချက်အလက်တွေက ဒီစာအုပ် ရေးသားချိန်အတိုင်း ဖြစ်ပါတယ်။ Tool တွေက အမြဲ update ဖြစ်နေတာမို့ မဆင်မခြင် ရိုက်မထည့်ခင် နောက်ဆုံး အခြေအနေကို [တရားဝင် documentation](#) မှာ အတည်ပြုပါ။

macOS / Linux မှာ — Terminal ဖွင့်ပြီး ဒီ Command ကို ရိုက်ပါ —

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows မှာ — PowerShell ဖွင့်ပြီး —


```
irm https://claude.ai/install.ps1 | iex
```

npm နဲ့ တင်ချင်ရင် (Node.js 18 နောက်ပိုင်း ရှိထားဖို့ လိုတယ်) —

```
npm install -g @anthropic-ai/claude-code
```

သတိပေးချက် တစ်ခု — npm နဲ့ တင်တဲ့အခါ `sudo` မသုံးပါနဲ့။ `sudo` ဆိုတာ "ကျုပ်က အိမ်ရှင်၊ ခွင့်ပြုချက် မလိုဘူး" ဆိုပြီး အတင်းဝင်တာမျိုးမို့ နောက်ပိုင်း Permission ပြဿနာတွေ တသိတတန်း လာတတ်တယ်။

တင်ပြီးရင် Terminal အသစ်ဖွင့်ပြီး စစ်ကြည့်ပါ —

```
claude --version
```

Version နံပါတ် ပေါ်လာရင် အောင်မြင်ပြီ။ `command not found` ဆိုပြီး ပြန်ပြောရင်တော့ စိတ်မပူပါနဲ့။ ကွန်ပျူတာလောကမှာ ပထမဆုံးအကြိမ်တည်းက အဆင်ပြေတာမျိုး ရှားတယ်။ Terminal ပိတ်ပြီး အသစ်ပြန်ဖွင့်ကြည့်ပါ။ `claude` command တော့တွေ့သွားပြီး တစ်ခြား error တွေ ပြနေရင် `claude doctor` ဆိုတဲ့ Command ရှိတယ်။ နာမည်အတိုင်းပဲ — ဆရာဝန်ခေါ်ပြကြည့်တာ။ ဘာကြောင့် မအီမသာ ဖြစ်နေလဲ သူက စစ်ဆေးပြီး ပြောပြပေးလိမ့်မယ်။

ပထမဆုံး တွေ့ဆုံခြင်း (first date) — စကားစမြည် ပြောကြည့်ခြင်း

ကဲ — စမ်းသပ်ဖို့ Folder တစ်ခု ဆောက်ပြီး Claude ကို ခေါ်ကြည့်ရအောင်။ (ဒါက စမ်းကြည့်ရုံသက်သက်ပါ — ဒေါ်မြရီဆိုင် တကယ့် Project ကိုတော့ အခန်း (၄) မှာ စဆောက်ပါမယ်။)

```
mkdir claude-test
cd claude-test
claude
```

ပထမဆုံးအကြိမ် Run ရင် Browser ပွင့်လာပြီး Login ဝင်ခိုင်းလိမ့်မယ်။ ဝင်ပြီးရင် Terminal ထဲမှာ Claude ရဲ့ စာရိုက်ကွက်လေး ပေါ်လာမယ်။ ဒီအချိန်ကစပြီး ကိုယ်ရိုက်သမျှကို Claude က ဖတ်ပြီး အလုပ်လုပ်ပေးတော့မယ်။

စမ်းကြည့်ချင်ရင် ဒီလိုလေး ရိုက်ကြည့်ပါ —

```
> မင်္ဂလာပါ။ ဒီ Folder ထဲမှာ ဘာဖိုင်တွေ ရှိလဲ ကြည့်ပေးပါ။
```

Folder အသစ်စက်စက်မို့ "ဘာမှ မရှိသေးပါဘူး ခင်ဗျာ" လို့ ပြန်ဖြေလိမ့်မယ်။ ဒါပေမယ့် ဒီတစ်ကြောင်းက အရေးကြီးတယ်။ **Claude Code က မြန်မာလို ပြောလို့ရတယ်** ဆိုတာ သိသွားပြီလေ။ ကုန်တွေကတော့ English နဲ့ ရေးမှာပေမယ့် ခိုင်းတာကတော့ ကိုယ်တတ်တဲ့ ဘာသာစကားနဲ့ ခိုင်းလို့ရတယ်။ ကြိုက်တာနဲ့သာခိုင်း။ English နဲ့ ခိုင်းနိုင်ရင် ပိုကောင်းတယ်။ မြန်မာလို ခိုင်းရင် စာလုံးရေများတာနဲ့ ဘာသာပြန်နေရတာနဲ့ token ပိုကုန်မယ်။

သိထားသင့်တဲ့ Slash Command များ

Claude Code ထဲမှာ `/` နဲ့ စတဲ့ Command လေးတွေ ရှိတယ်။ အားလုံး မှတ်စရာ မလိုဘူး။ ဒီလေးခုလောက် သိရင် လောလောဆယ် လုံလောက်တယ် —

Command	အလုပ်
<code>/help</code>	ဘာတွေ လုပ်လို့ရလဲ ပြပေးတယ်
<code>/init</code>	Project ကို လေ့လာပြီး CLAUDE.md ဖိုင် ဆောက်ပေးတယ် (နောက်အခန်းမှာ ရှင်းမယ်)
<code>/clear</code>	စကားပိုင်း ရှင်းပစ်တယ် — ခေါင်းရှင်းသွားအောင်
<code>/exit</code>	Claude Code ကနေ ထွက်တယ်

Permission — ခွင့်ပြုချက် ယဉ်ကျေးမှု

Claude Code ရဲ့ သဘောကောင်းတဲ့ အချက်တစ်ခုက **မိုင်ပြင်တွေမယ်၊ Command Run တွေမယ်ဆိုရင် ကိုယ့်ကို အရင်ခွင့်တောင်းတယ်။** "ဒီမိုင်ကို ဒီလိုပြင်မယ် — ရမလား?" ဆိုပြီး ပြမယ်။ ကိုယ်က Yes/No ပြန်ဖြေရမယ်။

Junior Developer တွေအတွက် အကြံပြုချင်တာက — **အစပိုင်းမှာ ဒီအတိုင်းပဲ ထားပါ။** Approve လုပ်ခိုင်းတိုင်း သူ ဘာလုပ်တော့မလဲ ဖတ်ကြည့်ပါ။ ကိုယ်မသိသေးတာဆိုရင် ဘာလို့ ဒီ command သုံးသွားလည်း ပြန်လေ့လာပါ။ ဒါဟာ ကိုယ့်အတွက် အခမဲ့ သင်ခန်းစာတွေ။ Agent ကို "ဘာမဆိုလုပ် ခွင့်မတောင်းနဲ့" ဆိုပြီး လွှတ်ပေးတဲ့ Mode တွေလည်း ရှိတယ်။ ဒါပေမယ့် ကားမောင်းသင်စ လူတစ်ယောက်ကို အဝေးပြေးလမ်းမပေါ် ပေးမောင်းပြီး ကိုယ်က မျက်လုံးမိတ်စီးရင် ဘာဖြစ်မလဲ တွေးကြည့်ပါ။

နောက်ထပ် အာမခံချက်တစ်ခုက **Git**။ Claude ကို အလုပ်တစ်ခုခု စမခိုင်းခင် Commit လုပ်ထားပါ။ Agent က တစ်ခုခုကို ဖရိုဖရဲ လုပ်သွားခဲ့ရင်တောင် `git checkout` တစ်ချက်နဲ့ အရင်အတိုင်း ပြန်ရောက်တယ်။ Git ဆိုတာ Agent ခေတ်ရဲ့ အသက်ကယ်အင်္ကျီ။ မဝတ်ဘဲ ရေထဲမဆင်းပါနဲ့။

နောက်ဆုံး သတိပေးချက်တစ်ခု — **လျှို့ဝှက်ချက်တွေ (API key၊ password၊ .env ဖိုင်ထဲက အချက်အလက်) ကို Prompt ထဲ တိုက်ရိုက် မကူးထည့်ပါနဲ့။** ဒါတွေက စကားပိုင်းမှတ်တမ်းထဲ ကျန်ခဲ့တတ်တယ်။ Git commit လုပ်တဲ့အခါလည်း `.env` လို လျှို့ဝှက်ဖိုင်တွေ မပါသွားအောင် `.gitignore` ထဲ ကြိုထည့်ထားပါ။

လက်တွေ့လုပ်ကြည့်ရန်

Claude Code ကို install လုပ်၊ `claude --version` နဲ့ စစ်၊ ပြီးရင် `claude-test` folder ထဲမှာ Claude ကို ဖွင့်ပြီး မြန်မာလို စကား တစ်ခွန်းနှစ်ခွန်း ပြောကြည့်ပါ။ `/help` ရိုက်ပြီး ဘာတွေ လုပ်လို့ရလဲ လျှောက်ကြည့်ပါ။ ဒီ folder ကို နောက်ပိုင်း ဖျက်ပစ်လို့ရတယ် — အခန်း (၄) မှာ တကယ့် Project ကို အသစ်ကနေ စဆောက်မှာမို့။

ကဲ — စက်ထဲမှာ Claude Code ရောက်နေပြီ။ နောက်အခန်းမှာ ဒီကောင်ကို ဘယ်လို ခိုင်းရင် ကောင်းကောင်း အလုပ်လုပ်လဲ ဆိုတဲ့ ခိုင်းနည်းခိုင်းဟန် ပညာကို ပြောပြမယ်။

← အခန်း (၁) · 🏠 မာတိကာ · အခန်း (၃) →

အခန်း (၃) — Agent ကို အလုပ်ခိုင်းတတ်အောင် သင်ခြင်း

ကျုပ်တို့ ရုပ်ကွက်ထဲမှာ ဦးဘသိန်းဆိုတာ ရှိတယ်။ သူ့ဇနီးက ဈေးဝယ်ခိုင်းရင် "ဟင်းသီးဟင်းရွက် ဝယ်ခဲ့" လို့ပဲ ပြောတယ်။ ဦးဘသိန်း ပြန်လာတိုင်း ရန်ဖြစ်ကြတယ်။ ဘာလို့လဲဆိုတော့ သူ့ဇနီး လိုချင်တာက ခရမ်းချဉ်သီး၊ ဦးဘသိန်း ဝယ်လာတာက ကန်စွန်းရွက်။ "ဟင်းသီးဟင်းရွက်ပဲ ပြောတာပဲ၊ ဘာမှားလို့လဲ" ဆိုပြီး ဦးဘသိန်းက မခံချင်။

Agent ကို ခိုင်းတာလည်း ဒီသဘောပဲ။ **ကိုယ်ပြောတဲ့အတိုင်း သူလုပ်တယ်။ ကိုယ်စိတ်ထဲက အတိုင်းတော့ မဟုတ်ဘူး။** Agent က စိတ်ဖတ်တတ်တဲ့ ဗေဒင်ဆရာ မဟုတ်ဘူး။ ဒါကြောင့် Agentic Coding ရဲ့ ပညာတစ်ဝက်က ကုဒ်ပညာ မဟုတ်ဘဲ **ပြောတတ်ဆိုတတ်တဲ့ ပညာ** ဖြစ်နေတာ။

CLAUDE.md — အိမ်ထောင်စုစာရင်း

Claude Code မှာ ထူးခြားချက်တစ်ခုက **CLAUDE.md** ဆိုတဲ့ ဖိုင်။ Project Folder ထဲမှာ ဒီနာမည်နဲ့ ဖိုင်ရှိရင် Claude က Session စတိုင်း ဒီဖိုင်ကို အရင်ဖတ်တယ်။

သဘောက ဒီလို — အိမ်မှာ အလုပ်သမားအသစ် ခန့်တိုင်း "ကျုပ်တို့အိမ်မှာ ဖိနပ်ချွတ်ပြီးမှ တက်ရတယ်၊ ရေခဲသေတ္တာထဲက မုန့်တွေက ကလေးတွေအတွက်၊ ခွေးကြီးက ကိုက်တတ်တယ်" ဆိုပြီး တစ်ခေါက်စီ ရှင်းပြနေရရင် ပင်ပန်းတယ်။ ဒီအချက်တွေကို စာရွက်ပေါ်ချရေးပြီး နံရံမှာ ကပ်ထားလိုက်ရင် ဘယ်အလုပ်သမား ရောက်ရောက် ဖတ်ပြီးသား ဖြစ်သွားတယ်။ CLAUDE.md ဆိုတာ ဒီနံရံကပ် စာရွက်ပဲ။

ကိုယ်တိုင် ရေးစရာတောင် မလိုဘူး။ Claude Code ထဲမှာ —

```
/init
```

လို့ ရိုက်လိုက်ရင် သူက Project တစ်ခုလုံးကို လျှောက်ကြည့်ပြီး CLAUDE.md ကို သူ့ဘာသာသူ ရေးပေးတယ်။ ရေးပြီးရင် ကိုယ်ဝင်ပြင်လို့ ရတယ်။ ထဲမှာ ဘာတွေ ပါသင့်လဲဆိုတော့ —

Myaree Store

Project အကြောင်း
ဒေါ်မြရီရဲ့ ကုန်စုံဆိုင်အတွက် mini ecommerce website

Tech Stack

- Frontend: React + Vite
- Styling: Tailwind CSS
- Data: products.json (Backend မရှိသေး)

စည်းကမ်းများ

- Component တွေကို function component နဲ့ ရေးရန်
- ကုဒ်ထဲ comment တွေကို မြန်မာလို ရေးရန်
- feature တစ်ခုပြီးတိုင်း npm test run ရန်

Commands

- `npm run dev` - dev server စတင်ရန်
- `npm test` - test များ run ရန်

CLAUDE.md ကောင်းကောင်းရေးထားတဲ့ Project မှာ Agent က ရေရေလည်လည် အလုပ်လုပ်တယ်။ မရှိတဲ့ Project မှာတော့ ဦးဘသိန်း ဈေးဝယ်သလို ဖြစ်တတ်တယ်။

မှတ်သားစရာ တစ်ခုက — CLAUDE.md ထဲမှာ ရေးတဲ့ စည်းကမ်းတွေက **တိတိကျကျ စစ်ဆေးလို့ရတာမျိုး** ဖြစ်ရမယ်။ "ကုဒ်ကို သေသေချာချာ ရေးပါ" ဆိုတာမျိုးက အလကား။ ဘယ် Agent မှ "ကျုပ် ပေါ့ပေါ့ဆဆ ရေးမယ်" လို့ ကြံရွယ်ပြီး ရေးတာ မဟုတ်ဘူး။ "Function တိုင်းမှာ Type ကြေညာပါ" ဆိုတာမျိုးကျမှ တကယ် အသုံးဝင်တာ။

Plan Mode — မဆောက်ခင် ပုံစံအရင်ကြည့်

Claude Code မှာ **Plan Mode** ဆိုတာ ရှိတယ်။ ဒီ Mode မှာ Claude က ကုဒ် မရေးသေးဘူး၊ ဖိုင် မပြင်သေးဘူး။ **ဘာတွေ လုပ်မယ်ဆိုတဲ့ အစီအစဉ်ကိုပဲ ချပြတယ်။** ကိုယ်က အစီအစဉ်ကို ကြည့်၊ ပြင်ချင်တာပြင်၊ သဘောတူမှ သူက စလုပ်တယ်။

အိမ်ဆောက်မယ့်သူက ပန်းရန်ဆရာကို "မနက်ဖြန် အုတ်စစ်" လို့ ချက်ချင်း မခိုင်းဘူးလေ။ အိမ်ပုံစံ အရင်ကြည့်တယ်။ အိပ်ခန်း ဘယ်နှခန်း၊ မီးဖိုချောင် ဘယ်ဘက်၊ အိမ်သာ ဘယ်နေရာ။ ပုံစံမှာ သဘောမတူရင် ခဲတံနဲ့ ဖျက်ပြင်လို့ရတယ်။ အုတ်စစ်ပြီးမှ ပြင်ရင်တော့ တူနဲ့ ထုရတော့တာပေါ့။

ကြီးမားတဲ့ အလုပ်၊ ဖိုင်များများ ထိမယ့် အလုပ်မျိုးဆိုရင် Plan Mode ကို အရင်သုံးပါ။ "ဒီ Feature ကို ဘယ်လို တည်ဆောက်မလဲ အစီအစဉ် အရင်ချပြပါ" လို့ ပြောရုံနဲ့လည်း ရတယ်။ ဒါက အချိန်ကုန်သလို ထင်ရပေမယ့် တကယ်တော့ အချိန် အကုန်သက်သာဆုံး နည်း။ လမ်းမှားပြီးမှ ပြန်လှည့်ရတာထက် လမ်းမထွက်ခင် မြေပုံကြည့်တာက မြန်တယ်။

Prompt ရေးနည်း — ခိုင်းတတ်သူ၏ အနုပညာ

ကောင်းတဲ့ Prompt နဲ့ ညံ့တဲ့ Prompt ကွာခြားပုံကို ဥပမာနဲ့ ကြည့်ရအောင်။

ညံ့တဲ့ Prompt:

> website လုပ်ပေး

ဒါက "ဟင်းသီးဟင်းရွက် ဝယ်ခွဲ" အဆင့်။ ရလာမယ့် Website က ဘာဖြစ်လာမလဲ ဘုရားသိ။

သင့်တင့်တဲ့ Prompt:

> ecommerce website တစ်ခု လုပ်ပေးပါ။ ပစ္စည်းတွေပြတဲ့ စာမျက်နှာနဲ့ ဈေးခြင်းတောင်း ပါရမယ်။

ဒါက "သီးစုံပဲကုလားဟင်းချက်ဖို့ လိုအပ်တာတွေဝယ်ခွဲ" အဆင့်။ နည်းနည်း ပိုကောင်းလာပြီ။

ကောင်းတဲ့ Prompt:

> React + Vite နဲ့ mini ecommerce တစ်ခု စဆောက်ပါမယ်။ ပထမအဆင့်အနေနဲ့ product list
> စာမျက်နှာ လုပ်ပေးပါ။ ပစ္စည်းအချက်အလက်တွေက src/data/products.json ထဲက ယူပါ။
> ပစ္စည်းတစ်ခုချင်းမှာ ပုံ၊ နာမည်၊ ဈေးနှုန်း (ကျပ်ငွေ)၊ "ခြင်းထဲထည့်မည်" ခလုတ် ပါရမယ်။
> Mobile မှာ တစ်တန်း၊ Desktop မှာ လေးတန်း ပြပါ။ ခလုတ်တော့ အလုပ်မလုပ်သေးလည်း ရတယ် -
> ခြင်းစနစ်ကို နောက်အဆင့်မှ လုပ်မယ်။

ဒါကျမှ "ခရမ်းသီး (ခရမ်းရောင်၊ အရှည်မျိုး) အလုံးကြီးကြီး တစ်ပိဿာ၊ ဈေးက သုံးထောင်ထက် မကျော်စေနဲ့" အဆင့် ရောက်တယ်။ ဒီလို Prompt မျိုးရရင် Agent က လမ်းမချော်တော့ဘူး။

ကောင်းတဲ့ Prompt ရဲ့ လက္ခဏာ လေးချက် —

1. **ဘာလိုချင်လဲ တိတိကျကျ** — "ကောင်းကောင်းလေး" မဟုတ်ဘဲ "ပုံ၊ နာမည်၊ ဈေးနှုန်း ပါရမယ်"
2. **ဘောင်သတ်မှတ်ခြင်း** — "ဈေးဝယ်ရင် မုန့်ဝင်မစားခွဲနဲ့" ဆိုတာမျိုး၊ မလုပ်ရမယ့်အရာ ပြောထား
3. **တစ်ကြိမ်တစ်ခု** — Website တစ်ခုလုံး တစ်ခါတည်း မခိုင်းဘဲ အဆင့်လိုက်ခွဲ
4. **အောင်မြင်မှုကို တိုင်းတာလို့ရအောင်** — "Mobile မှာ တစ်တန်း" ဆို မှန်မမှန် ကြည့်လို့ရတယ်

အဆင့်လိုက်ခွဲခြင်း — Agentic Coding ရဲ့ အသည်းနှလုံး

နောက်ဆုံးအချက်ကို ထပ်လေးနက်ချင်တယ်။ Agent ကို ခိုင်းတဲ့အခါ **ဆင်တစ်ကောင်လုံး တစ်ခါတည်း မမျိုခိုင်းပါနဲ့။** ဆင်ကို စားချင်ရင် တစ်လုတ်ချင်း လှီးရတယ်။

"Ecommerce website အပြည့်အစုံ လုပ်ပေး" လို့ တစ်ကြောင်းတည်း ခိုင်းလိုက်ရင် Agent က ကြိုးစားပြီး လုပ်ပေးပါလိမ့်မယ်။ ဒါပေမယ့် ဖိုင် သုံးလေးဆယ် တစ်ပြိုင်နက် ထွက်လာပြီး ကိုယ်က ဘာမှန်းညာမှန်း မသိတော့ဘူး။ တစ်ခုခု မှားရင်လည်း ဘယ်နားက စမှားလဲ ရှာမတွေ့တော့ဘူး။ ဒါက Junior Developer အတွက် အန္တရာယ် အကြီးဆုံး တွင်းပဲ။

ဒီအစား —

- အဆင့် ၁ — Project setup
- အဆင့် ၂ — Product list ပြခြင်း
- အဆင့် ၃ — ခြင်းထဲထည့်ခြင်း
- အဆင့် ၄ — Checkout

ဆိုပြီး တစ်ဆင့်ချင်း ခိုင်း၊ တစ်ဆင့်ပြီးတိုင်း **ကိုယ်တိုင် Run ကြည့်၊ ကုဒ်ဖတ်ကြည့်၊ Git Commit လုပ်၊** ပြီးမှ နောက်တစ်ဆင့် သွားပါ။ ဒီအလေ့အကျင့်က နောက်အခန်းတွေမှာ လက်တွေ့မြင်ရလိမ့်မယ်။

လက်တွေ့လုပ်ကြည့်ရန်

အခန်း (၁) မှာ ချရေးခဲ့တဲ့ အလုပ်ကို ပြန်ယူပါ။ အခု "ညှိတဲ့ Prompt → ကောင်းတဲ့ Prompt" အဆင့်တက်တဲ့ ပုံစံ အတိုင်း၊ အဲဒီအလုပ်ရဲ့ ပထမဆင့်အတွက် **ကောင်းတဲ့ Prompt တစ်ခု** ရေးကြည့်ပါ — လိုချင်တာ တိတိကျကျ၊ ဘောင်သတ်၊ အောင်မြင်မှုကို တိုင်းတာလို့ရအောင် (လက္ခဏာ လေးချက်နဲ့ ကိုက်မကိုက် ပြန်စစ်ပါ)။

ကဲ — သီအိုရီတွေ တော်လောက်ပြီ။ နောက်အခန်းမှာ ဒေါ်မြရီအတွက် ဆိုင်စဆောက်ကြမယ်။

← အခန်း (၂) · 🏠 မာတိကာ · အခန်း (၄) →

အခန်း (၄) — Mini Ecommerce Project စတင်ခြင်း (ဒေါ်မြရီ ကုန်စုံဆိုင် Online တက်မည်)

ကျုပ်တို့ရပ်ကွက်ထဲက ဒေါ်မြရီဆိုတာ သူ့အိမ်ရှေ့မှာ နှစ်ပေါင်းနှစ်ဆယ် ဈေးရောင်းလာတဲ့ ပုဂ္ဂိုလ်။ ရောင်းတာက ချောကလက်၊ မုန့်၊ ဆပ်ပြာ၊ ဓာတ်ခဲ — အိမ်သုံးပစ္စည်း စုံတယ်။ တစ်နေ့တော့ ဒေါ်မြရီက ကျုပ်ကို လှမ်းခေါ်တယ်။

"မောင်ရင်က ကွန်ပျူတာသမား ဆိုတော့ — ငါ့ဆိုင်လေးကို အွန်လိုင်းပေါ်တင်ပေးလို့ ရမလား။ မြေးမလေးက ပြောတယ်၊ ခုခေတ် ဆိုင်ဆိုတာ ဖုန်းထဲမှာ ရှိရတယ်တဲ့။"

ကျုပ်လည်း "ရပါတယ် ဒေါ်ဒေါ်" လို့ ဖြေလိုက်မိတယ်။ ပြောပြီးမှ သတိရတယ်။ ကျုပ်က Junior Developer။ Ecommerce Website ဆိုတာ တစ်ခါမှ မဆောက်ဖူးဘူး။ ဒါပေမယ့် ကိစ္စမရှိဘူး။ **ကျုပ်မှာ Agent ရှိတယ်။** ဒီအခန်းက အဲဒီနေ့က ကျုပ်လုပ်ခဲ့သမျှကို ပြန်ပြောပြတာပဲ။

အဆုံးမှာ ဘာရလာမလဲ ကြိုမြင်ထားရအောင် — ဆောက်မယ့် မြရီစတိုးရဲ့ ဝယ်သူ ခရီးစဉ်က ဒီလို —

[ပစ္စည်းစာရင်း] → [ဈေးခြင်းတောင်း] → [Checkout Form] → [Order Summary] → [Viber ပို့]
Product List Cart (နာမည်/ဖုန်း/လိပ်စာ) (Screenshot ရိုက်) ဒေါ်မြရီဆီ ရောက်

အခန်း (၄) မှာ ပထမနှစ်ခန်း၊ အခန်း (၅) မှာ ကျန်တာ တစ်ဆင့်ချင်း ဆောက်သွားပါမယ်။

အဆင့် (၁) — မဆောက်ခင် စဉ်းစား

Claude Code ကို မဖွင့်ခင် ကျုပ် စာရွက်ပေါ်မှာ အရင်ချရေးတယ်။ ဒေါ်မြရီဆိုင် Website မှာ ဘာတွေ လိုမလဲ —

- ပစ္စည်းစာရင်း ပြတဲ့ စာမျက်နှာ (ပုံ၊ နာမည်၊ ဈေး)
- ဈေးခြင်းတောင်း (Cart) — ထည့်လို့ရ၊ ဖြုတ်လို့ရ
- Checkout — ဝယ်သူနာမည်၊ ဖုန်း၊ လိပ်စာ ဖြည့်ပြီး Order တင်တာ
- Backend မလိုသေး — ဒေါ်မြရီမှာ Server ငှားဖို့ ပိုက်ဆံ မရှိသေးဘူး။ Order ကျရင် ဖုန်းဆက်တဲ့ စနစ်နဲ့ပဲ စမယ်။

ဒီစာရွက်က ပေါ့သေးသေး မဟုတ်ဘူး။ **Agent ကို ခိုင်းတဲ့အခါ ကိုယ့်ခေါင်းထဲမှာ မရှင်းတာကို Agent ကလည်း ရှင်းအောင် လုပ်ပေးနိုင်ဘူး။** ကိုယ်တောင် ဘာလိုချင်မှန်း မသိရင် Agent က ဘာလုပ်ပေးရမှန်း သိပါ့မလား။

အဆင့် (၂) — Project Setup ခိုင်းခြင်း

```
mkdir myaree-store
cd myaree-store
git init
claude
```

Claude Code ပွင့်လာတော့ ပထမဆုံး Prompt —

```
> React + Vite + Tailwind CSS နဲ့ project အသစ် setup လုပ်ပေးပါ။ TypeScript မသုံးသေးဘူး၊
> ရိုးရိုး JavaScript နဲ့။ Setup ပြီးရင် dev server run လို့ရကြောင်း စစ်ပြပါ။
```

ဒီနေရာမှာ TypeScript မသုံးဘူးလို့ ကျုပ်ပြောလိုက်တာ သတိထားမိလား။ TypeScript က ကောင်းပါတယ်။ ဒါပေမယ့် ကျုပ်ရဲ့ရည်ရွယ်ချက်က Agentic Coding သင်တာ၊ Type စနစ်နဲ့ နှုတ်လုံးတာ မဟုတ်ဘူး။ **ကိုယ်နားလည်နိုင်တဲ့ Tech ကို ရွေးပါ။ Agent က ရေးပေးမှာ ဖြစ်ပေမယ့် ဖတ်ရမှာက ကိုယ်။**

Claude က `npm create vite@latest` တို့ ဘာတို့ Run မယ်၊ Tailwind ထည့်မယ်၊ တစ်ခုချင်း ခွင့်တောင်းလိမ့်မယ်။ ကျုပ်က Approve နှိပ်ရင်း သူ့ဘာတွေလုပ်နေလဲ ဘေးကနေ ခိုးကြည့်တယ်။ ဒါက အလကားရတဲ့ ကျူရှင်။ Setup ပြီးတော့ —

```
✓ Dev server started at http://localhost:5173
```

Browser ထဲမှာ Vite ရဲ့ မျက်နှာစာ ပေါ်လာတယ်။ ဒီအချိန်မှာ ချက်ချင်းလုပ်ရမှာ —

```
git add .
git commit -m "project setup"
```

တစ်ဆင့်ပြီးတိုင်း Commit။ ဒါ ဘုရားစာလို ရွတ်ထားပါ။

(`git add .` လုပ်လိုက်ရင် `node_modules/` ဖိုလ်ဒါကြီး ပါသွားမလားလို့ စိုးရိမ်စရာ မလိုပါဘူး။ Vite က `.gitignore` ဖိုလ်ဒါကို ကြိုဆောက်ပေးထားပြီး `node_modules/` ကို ဖယ်ထားပြီးသားပါ။)

အဆင့် (၃) — CLAUDE.md ဆောက်ခြင်း

```
> /init
```

Claude က Project ကို လေ့လာပြီး CLAUDE.md ရေးပေးတယ်။ ရေးပြီးတာကို ကျုပ်ဝင်ပြင်ပြီး ဒီအချက်တွေ ဖြည့်တယ် —

```
## Project အကြောင်း
ဒေါ်မြရီ ကုန်စုံဆိုင်အတွက် mini ecommerce Backend မရှိ။
Order data ကို localStorage ထဲ သိမ်းမယ်။
```

```
## စည်းကမ်းများ
- ဈေးနှုန်းအားလုံး မြန်မာကျပ်ငွေ။ ဖော်မတ် - ၃,၅၀၀ ကျပ်
- UI စာသားတွေ မြန်မာလို ရေးရန်
- Component တစ်ခုကို ဖိုင်တစ်ဖိုင်၊ src/components/ ထဲမှာ
- feature ပြီးတိုင်း npm run dev နဲ့ error မရှိကြောင်း စစ်ရန်
```

အဆင့် (၄) — Product Data နဲ့ Product List

ဒေါ်မြရီဆီက ပစ္စည်း ဆယ်မျိုးလောက် စာရင်းကောက်လာပြီး —

```
> src/data/products.json ဖိုင် ဆောက်ပေးပါ။ ပစ္စည်း ၁၀ ခု - တစ်ခုချင်းမှာ id, name (မြန်မာလို),
> price (ကျပ်), image, category ပါရမယ်။ ဥပမာ ပစ္စည်းတွေက - ရွှေစီချောကလက် ၅၀၀ ကျပ်၊
> လက်ဖက်ရည်ဆား တစ်ထုပ် ၁,၂၀၀ ကျပ်၊ ဆပ်ပြာ ၈၀၀ ကျပ်... ။ ပုံတွေအတွက်တော့
> placeholder ပုံတွေပဲ သုံးထားပါ။
>
> ပြီးရင် ProductList component ဆောက်ပါ။ products.json ကို ဖတ်ပြီး Card ပုံစံပြပါ။
> Card ထဲမှာ ပုံ၊ နာမည်၊ ဈေးနှုန်း၊ "ခြင်းထဲထည့်မည်" ခလုတ်။ ခလုတ်က အခုတော့
> console.log ပဲ လုပ်ထားပါ။ Mobile မှာ ၂ တန်း၊ Desktop မှာ ၄ တန်း။
```

မိနစ်အနည်းငယ်အတွင်း products.json ၊ ProductCard.jsx ၊ ProductList.jsx ဆိုပြီး ဖိုင်တွေ ထွက်လာတယ်။ Browser မှာကြည့်တော့ ဒေါ်မြရီရဲ့ ပစ္စည်းတွေ Card လေးတွေနဲ့ သပ်သပ်ရပ်ရပ်။ အိမ်ရှေ့ပလက်ဖောင်းပေါ်က ဆိုင်ခင်းတာ နှစ်ပေါင်းနှစ်ဆယ်ကြာတဲ့ ဆိုင်၊ အွန်လိုင်းပေါ်ဆိုင်ခင်းတာ ဆယ်မိနစ်။

အဆင့် (၅) — ကုန်ကို ပြန်ဖတ် (ဒီအဆင့် ကျော်ရင် အပြစ်ရှိ၏)

ဒီနေရာမှာ Junior Developer အများစု လုပ်တတ်တဲ့ အမှားက — အလုပ်ဖြစ်တာ မြင်ရပြီဆိုတော့ ကုန်မဖတ်တော့ဘဲ ရှေ့ဆက်တာ။ ကျုပ်ကတော့ Claude ကိုပဲ ပြန်ခိုင်းတယ် —

```
> ProductList.jsx ထဲက ကုန်ကို လိုင်းချင်းလိုက် ရှင်းပြပါ။ map function က ဘာလုပ်လဲ၊
> key prop က ဘာလို့ လိုလဲ ဆိုတာပါ ရှင်းပါ။
```

ဒါက Agentic Coding ရဲ့ လျှို့ဝှက်ချက်တစ်ခု — **Agent ဟာ ကုန်ရေးစက်တင် မဟုတ်ဘူး၊ ကိုယ်ပိုင်ဆရာလည်း ဖြစ်တယ်။** သူရေးထားတာကို သူ့ကိုပဲ ရှင်းခိုင်း။ နားမလည်သေးရင် ထပ်မေး။ ဒီဆရာက ဘယ်တော့မှ စိတ်မတိုဘူး၊ ကျူရှင်ခ ထပ်မတောင်းဘူး (ဖုန်းဘေ (token ဖိုး)တော့ ကုန်တာပေါ့)။

နားလည်ပြီဆိုမှ —

```
git add .
git commit -m "product list ပြီး"
```

လက်တွေ့လုပ်ကြည့်ရန်

ကိုယ်တိုင် Setup ကနေ Product List အထိ လိုက်ဆောင်ကြည့်ပါ။ ပြီးရင် ဒေါ်မြရီဆိုင်အစား ကိုယ်သိတဲ့ ဆိုင် တစ်ဆိုင် (ဥပမာ — အနီးက စားသောက်ဆိုင်၊ စာအုပ်ဆိုင်) ကို ကိုယ်စားထားပြီး `products.json` ထဲက ပစ္စည်းတွေကို ကိုယ့်ဆိုင်အတွက် ပြောင်းရေးကြည့်ပါ။ Agent က ကုန်ဖွဲ့စည်းပုံ တူတူနဲ့ အလုပ်လုပ်နိုင်မလား စမ်းကြည့်တာပါ။

နောက်အခန်းမှာ ဈေးခြင်းတောင်းနဲ့ Checkout — တကယ့် ရျာနယ်လမ်းခွဲ အပိုင်းတွေ ဆက်ဆောင်ကြမယ်။

← အခန်း (၃) · 🏠 မာတိကာ · အခန်း (၅) →

အခန်း (၅) – Feature တွေ တစ်ခုချင်း ဆောက်ခြင်း

ဈေးခြင်းတောင်း – Ecommerce ရဲ့ နှလုံးသား

ဈေးခြင်းတောင်း (Shopping Cart) ဆိုတာ Ecommerce ရဲ့ နှလုံးသားပဲ။ ဒေါ်မြရီဆိုင်မှာဆို ဝယ်သူက ပစ္စည်း ကောက်ပြီး ဒေါ်မြရီလက်ထဲ ထည့်ထည့်ပေးတယ်။ ဒေါ်မြရီက ခေါင်းထဲမှာ ပေါင်းနေတယ်။ "ချောကလက် နှစ်ခု တစ်ထောင်၊ ဆပ်ပြာ ရှစ်ရာ၊ စုစုပေါင်း တစ်ထောင်ရှစ်ရာ" – ဒေါ်မြရီ ခေါင်းက ကွန်ပျူတာထက် မြန်တယ်။ ကျုပ်တို့ Website ကတော့ State Management နဲ့ လုပ်ရမယ်။

ဒီအကြောင်း Claude ကို တန်းမခိုင်းသေးဘဲ **အရင်ဆွေးနွေးတယ်။** ဒါလည်း Agentic Coding ရဲ့ ကောင်းတဲ့ အလေ့အကျင့် – Agent ကို အလုပ်သမားအဖြစ်တင် မသုံးဘဲ တိုင်ပင်ဖော် အဖြစ်လည်း သုံးတာ။

- > Cart state ကို ဘယ်လို manage လုပ်ရင် ကောင်းမလဲ။ ကျုပ်က React beginner ဖြစ်တယ်။
- > Redux တို့ Zustand တို့ ကြားဖူးတယ်၊ မသုံးဖူးဘူး။ ဒီ project အရွယ်အတွက် ရွေးချယ်စရာတွေကို
- > အားသာချက် အားနည်းချက်နဲ့ နှိုင်းယှဉ်ပြပါ။ ကုန်တော့ မရေးသေးပါနဲ့။

Claude က ရွေးချယ်စရာ သုံးမျိုး ချပြတယ် – Context API၊ Zustand၊ Redux။ ပြီးတော့ ဒီ Project အရွယ်ဆို Context API နဲ့ လုံလောက်ကြောင်း အကြံပေးတယ်။ "ကုန်မရေးသေးပါနဲ့" ဆိုတဲ့ စာကြောင်းလေး သတိထားမိ လား။ ဒါမပါရင် Agent ဆိုတာ စေတနာအရမ်းကြီးတော့ မေးရုံမေးတာနဲ့ ကုန်ဖိုင် ငါးဖိုင်လောက် ဆောက်ပြီးသား ဖြစ်တတ်တယ်။ စေတနာကြီးတဲ့ ဧည့်သည်က အိမ်ရှင် မခိုင်းဘဲ ထမင်းဟင်း ဝင်ချက်တာမျိုး – ရည်ရွယ်ချက် ကောင်းပေမယ့် မီးဖိုချောင် ပျက်တတ်တယ်။

Plan အရင်တောင်း၊ ပြီးမှ ဆောက်ခိုင်း

- > ကောင်းပြီ။ Context API နဲ့ပဲ သွားမယ်။ Cart feature တစ်ခုလုံးအတွက် Implementation plan
- > အရင်ချပြပါ – ဘယ်ဖိုင်တွေ အသစ်ဆောက်မလဲ၊ ဘယ်ဖိုင်တွေ ပြင်မလဲ၊ ဘာ function တွေ ပါမလဲ။

Claude ချပြတဲ့ Plan –

1. src/context/CartContext.jsx – cart state နဲ့ addToCart, removeFromCart, updateQuantity, clearCart functions
2. App.jsx ကို CartProvider နဲ့ ပတ်မယ်
3. ProductCard.jsx – ခလုတ်ကို addToCart နဲ့ ချိတ်မယ်
4. src/components/Cart.jsx – ခြင်းထဲက ပစ္စည်းစာရင်း၊ အရေအတွက် +/- ၊ စုစုပေါင်း
5. Header မှာ cart icon နဲ့ ပစ္စည်းအရေအတွက် badge

ဖတ်ကြည့်တယ်။ သဘောကျတယ်။ တစ်ခုပဲ ဖြည့်ချင်တယ် –

- > Plan ကောင်းတယ်။ တစ်ခုထပ်ထည့်ပါ – cart data ကို localStorage ထဲ သိမ်းပါ။
- > ဝယ်သူက Browser ပိတ်ပြီး ပြန်ဖွင့်ရင် ခြင်းထဲက ပစ္စည်းတွေ မပျောက်စေရဘူး။
- > ကဲ – ဒီ plan အတိုင်း ဆောက်ပါ။

ဒီနေရာမှာ သတိထားမိစရာ — **Plan ကို ကိုယ်က ပြင်ခွင့်ရှိတယ်။** Agent ချပြတဲ့ Plan ဆိုတာ အဆိုပြုချက်ပဲ၊ အမိန့်မဟုတ်ဘူး။ လက်မှတ်မထိုးခင် စာချုပ်ဖတ်သလို ဖတ်၊ ပြင်ချင်တာပြင်၊ ပြီးမှ လက်မှတ်ထိုး။

ဆယ့်ငါးမိနစ်လောက်အကြာမှာ Cart စနစ် ပြီးသွားတယ်။ Browser မှာ စမ်းကြည့်တယ် — ချောကလက် နှစ်ခု ထည့်၊ ခြင်းထဲမှာ နှစ်ခုပြ၊ တစ်ခုဖြုတ်၊ တစ်ခုပဲကျန်၊ Browser ပိတ်ပြီးပြန်ဖွင့်၊ ပစ္စည်းတွေ ရှိနေဆဲ။

```
git commit -am "cart စနစ်ပြီး"
```

Checkout — ဒေါ်မြရီနဲ့ ချိတ်ဆက်ခြင်း

နောက်ဆုံး Feature က Checkout။ ဒီမှာ လက်တွေ့ဆန်တဲ့ ပြဿနာတစ်ခု ရှိတယ်။ ဒေါ်မြရီမှာ Bank Account မရှိဘူး၊ Payment Gateway ဆိုတာ ဝေလာဝေး။ ဒေါ်မြရီ သုံးတတ်တာ ဖုန်းတစ်လုံးတည်း။ ဒါကြောင့် ကျုပ်တို့ Checkout က ခေတ်မီနည်းပညာနဲ့ မြန်မာနည်း မြန်မာဟန် ပေါင်းစပ်ထားတဲ့ ပုံစံ —

- > Checkout feature ဆောက်ပါ။ Flow က ဒီလို —
- > ၁။ Cart စာမျက်နှာမှာ "မှာယူမည်" ခလုတ်
- > ၂။ နှိပ်ရင် Form ပေါ်မယ် — ဝယ်သူနာမည်၊ ဖုန်းနံပါတ်၊ ပို့ဆောင်ရမယ့် လိပ်စာ
- > ၃။ ဖုန်းနံပါတ်က မြန်မာဖုန်းနံပါတ် format (၀၉ နဲ့စ၊ ဂဏန်း ၉-၁၁ လုံး) စစ်ပါ
- > ၄။ "အတည်ပြုမည်" နှိပ်ရင် order summary ပြပြီး order ကို localStorage ထဲ သိမ်းပါ
- > ၅။ ပြီးရင် "ကျေးဇူးတင်ပါတယ်၊ ဒေါ်မြရီက ဖုန်းပြန်ဆက်ပါလိမ့်မယ်" စာမျက်နှာ ပြပါ
- > ၆။ cart ကို ရှင်းပစ်ပါ

Prompt ကို ကြည့်ပါ။ Flow ကို နံပါတ်စဉ်နဲ့ ရေးပြထားတယ်။ **Agent ကို ခိုင်းတဲ့အခါ ကိုယ့်ခေါင်းထဲက မြင်ကွင်းကို စာနဲ့ ပုံဖော်ပြနိုင်လေ၊ ရလဒ်က ကိုယ်လိုချင်တာနဲ့ နီးလေ။**

Claude က CheckoutForm ဆောက်တယ်၊ Validation ရေးတယ်၊ ThankYou စာမျက်နှာ လုပ်တယ်။ စမ်းကြည့်တော့ တစ်ခုတော့ တွေ့တယ် — ဖုန်းနံပါတ်နေရာမှာ မြန်မာဂဏန်း "၀၉..." နဲ့ ရိုက်ရင် Form က လက်မခံဘူး။ ဒေါ်မြရီရဲ့ ဝယ်သူတွေထဲမှာ မြန်မာဂဏန်းနဲ့ ရိုက်မယ့်သူ သေချာပေါက်ပါမယ်။

- > ဖုန်းနံပါတ် field မှာ မြန်မာဂဏန်း (၀-၉) နဲ့ ရိုက်ရင်လည်း လက်ခံပါ။
- > အာရဗီဂဏန်းအဖြစ် အလိုအလျောက် ပြောင်းပေးပြီးမှ validate လုပ်ပါ။

ဒီလို ပြဿနာမျိုးက စာအုပ်ထဲမှာ မပါဘူး။ **လက်တွေ့သုံးမယ့်သူကို မြင်ယောင်ကြည့်မှ တွေ့တဲ့ ပြဿနာ။** Agent က ကုန်ကျခံတယ်၊ ဒါပေမယ့် ဒေါ်မြရီရဲ့ ဝယ်သူတွေကို သူမသိဘူး။ သိတာက ကိုယ်။ ဒါက Agent ခေတ်မှာ လူသားတန်ဖိုး ကျန်နေသေးတဲ့ နေရာ။

```
git commit -am "checkout ပြီး"
```

မြေးမလေးရဲ့ မေးခွန်း — ကျုပ် မျက်နှာပူသွားခြင်း

Checkout ပြီးလို့ ဂုဏ်ယူဝင်ကြွားစွာနဲ့ ဒေါ်မြရဲ့မြေးမလေးကို အရင်ပြကြည့်တယ်။ မြေးမလေးက ဖုန်းထဲမှာ စမ်းမှာကြည့်တယ်။ Form ဖြည့်တယ်။ အတည်ပြုတယ်။ "ကျေးဇူးတင်ပါတယ်" စာမျက်နှာ မြင်တယ်။ ပြီးတော့ ခေါင်းလေးမော့ပြီး မေးတယ် —

"ဦးလေး... ဒီ Order က ဘွားဘွားဆီ ဘယ်လိုရောက်မှာလဲ။"

ကျုပ် ငြိမ်သွားတယ်။ ခဏကြာအောင် ငြိမ်သွားတယ်။ ပြန်စဉ်းစားကြည့်တော့ — Order ကို localStorage ထဲ သိမ်းထားတယ်။ **localStorage ဆိုတာ ဝယ်သူရဲ့ Browser ထဲမှာ ရှိတာ။** ဝယ်သူဖုန်းထဲက ဗီရိုပေါ့။ ဆိုတော့ ဝယ်သူက Order တင်လိုက်တာ — သူ့စာကို သူ့ဗီရိုထဲ ရေးသိမ်းလိုက်တာနဲ့ အတူတူပဲ။ ဒေါ်မြရဲ့က တစ်ဖက်ရပ်ကွက်မှာ။ ဗီရိုသော့ကလည်း ဝယ်သူဆီမှာ။ "ဒေါ်မြရဲ့က ဖုန်းပြန်ဆက်ပါလိမ့်မယ်" ဆိုတဲ့ စာသားက **ဒေါ်မြရဲ့ ဘယ်တော့မှ မမြင်ရမယ့် Order အတွက် ပေးထားတဲ့ ကတိ။**

ဒီနေရာမှာ သတိထားစရာက — **Agent က ဘာမှ မမှားဘူး။** ကျုပ်ခိုင်းတဲ့အတိုင်း တစ်သဝေမတိမ်း၊ သပ်သပ်ရပ်ရပ် လုပ်ပေးတာ။ မှားတာက ကျုပ် Design။ Agent ဆိုတာ ခိုင်းတဲ့အတိုင်းလုပ်တဲ့ သဘောကောင်းလေးမို့ **မှားနေတဲ့ အကြံကိုလည်း ချောချောမွေ့မွေ့ အကောင်အထည်ဖော်ပေးတယ်။** "ဆရာ... ဒီ Order က ဒေါ်မြရဲ့ဆီ ရောက်မှာ မဟုတ်ဘူးနော်" လို့ တစ်ခါတလေ သတိပေးချင်ပေးမယ်။ ဒါပေမယ့် အားကိုးလို့ မရဘူး။ **စနစ်တစ်ခုလုံးရဲ့ အဓိပ္ပာယ်ရှိမရှိ စစ်ရမှာက ပိုင်ရှင်တာဝန်။** ကုဒ်မှန်တိုင်း စနစ်မှန်တယ်လို့ မဆိုနိုင်ဘူး။

ပြင်နည်း — Screenshot နဲ့ Viber

Backend ဆောက်ရင် ပြီးတာပဲလို့ ပြောချင်ပြောမယ်။ ဒါပေမယ့် ဒေါ်မြရဲ့မှာ Server ငှားဖို့ ပိုက်ဆံမရှိသေးဘူးဆိုတဲ့ ကန့်သတ်ချက်က ရှိနေဆဲ။ ဒီတော့ ကျုပ် ဒေါ်မြရဲ့ရဲ့ လက်ရှိဘဝကို ပြန်ကြည့်တယ်။ ဒေါ်မြရဲ့က ဖုန်းထဲမှာ ဘာအကျွမ်းကျင်ဆုံးလဲ — **Viber။** မြေးတွေနဲ့ Viber နဲ့ပြော၊ ဆွေမျိုးတွေဆီက ပုံတွေ Viber နဲ့လက်ခံ၊ တစ်နေကုန် Viber ထဲမှာ။ ဒါဆို ဖြေရှင်းနည်းက ရှင်းနေပြီ — **Order ကို ဒေါ်မြရဲ့ရဲ့ Viber ထဲ ရောက်အောင် ပို့ရမယ်။**

- > Checkout flow ကို ပြင်မယ်။ အခု design အတိုင်းဆို order က ဝယ်သူ browser ရဲ့
- > localStorage ထဲမှာပဲ ကျန်နေပြီး ဆိုင်ရှင်ဆီ မရောက်ဘူး။ ဒီလိုပြင်ပါ —
- > ၁။ "အတည်ပြုမည်" နှိပ်ပြီးရင် Order Summary စာမျက်နှာကို Screenshot ရိုက်ဖို့
- > သင့်တော်အောင် design လုပ်ပါ — order နံပါတ်၊ ရက်စွဲ၊ ပစ္စည်းစာရင်း၊ စုစုပေါင်း၊
- > ဝယ်သူနာမည်၊ ဖုန်း၊ လိပ်စာ — အားလုံး screen တစ်ခုတည်းမှာ ကြည့်လို့ရအောင်၊
- > စာလုံးကြီးကြီး ရှင်းရှင်း
- > ၂။ အောက်မှာ ညွှန်ကြားချက် ပြပါ — "ဒီစာမျက်နှာကို Screenshot ရိုက်ပြီး
- > ဒေါ်မြရဲ့ရဲ့ Viber (09xxxxxxxx) ကို ပို့ပေးပါ"
- > ၃။ "Viber ဖွင့်မည်" ခလုတ် ထည့်ပါ — `viber://chat?number=%2B959xxxxxxxx`
- > deep link နဲ့ ဒေါ်မြရဲ့ရဲ့ Viber chat တန်းပွင့်အောင်
- > ၄။ localStorage ထဲ သိမ်းတာကတော့ ဆက်သိမ်းပါ — ဝယ်သူဘက်က "ကျွန်တော့်
- > order history" ပြန်ကြည့်လို့ရအောင် သုံးမယ်

Claude က ပြင်ပေးတယ်။ အခု Flow အသစ်က — ဝယ်သူ အတည်ပြုတာနဲ့ လှလှပပ Order Summary ပေါ်လာမယ်။ Screenshot ရိုက်၊ "Viber ဖွင့်မည်" နှိပ်၊ ဒေါ်မြရီ Chat ပွင့်လာ၊ ပုံပို့ — ပြီးပြီ။ ဒေါ်မြရီဘက်ကလည်း ဘာအသစ်မှ သင်စရာမလိုဘူး။ Viber ထဲ ပုံဝင်လာရင် ဖွင့်ကြည့်တတ်တာ ဒေါ်မြရီရဲ့ မူလကျွမ်းကျင်မှုထဲမှာ ပါပြီးသား။ နည်းပညာအမြင့်ဆုံး ဖြေရှင်းနည်း မဟုတ်ပေမယ့် **သုံးမယ့်သူနဲ့ အကိုက်ညီဆုံး ဖြေရှင်းနည်း**။ Junior Developer တွေ မှတ်ထားသင့်တာက — ဝယ်သူအတွက် အလုပ်ဖြစ်တဲ့ ရိုးရိုးစနစ်က ဝယ်သူ နားမလည်တဲ့ ခမ်းနားစနစ်ထက် တန်ဖိုးရှိတယ်။

တစ်ခုတော့ ရိုးရိုးသားသား ပြောထားချင်တယ် — `viber://` deep link က ဖုန်းအမျိုးအစား၊ Viber version အလိုက် အမြဲတမ်း အလုပ်လုပ်ချင်မှ လုပ်တယ်။ တချို့ဖုန်းမှာ ဒေါ်မြရီရဲ့ Chat တန်းမပွင့်ဘဲ Viber app ကိုပဲ ဖွင့်ပေးတာမျိုး ဖြစ်နိုင်တယ်။ ဒါကြောင့် **တကယ့် အာမခံချက်က "Screenshot ရိုက်ပြီး ဒီနံပါတ်ကို ပို့ပါ" ဆိုတဲ့ ညွှန်ကြားချက်ပဲ** — deep link ခလုတ်ကတော့ အဆင်ပြေရင် ပိုမြန်အောင် ထည့်ပေးထားတဲ့ အပိုသာ။ ကိုယ် ကိုယ်တိုင် မထိန်းချုပ်နိုင်တဲ့ ပြင်ပ app (Viber, Facebook စသဖြင့်) အပေါ်မှီခိုတဲ့အခါတိုင်း "အဆင်ပြေရင် ကောင်း၊ မပြေရင်လည်း အလုပ်ဖြစ်နေအောင်" ဆိုတဲ့ ဒုတိယလမ်းကြောင်း အမြဲ ချန်ထားပေးရတယ်။

```
git commit -am "checkout ကို viber flow နဲ့ပြင် - မြရီစတိုး version 1"
```

ဒေါ်မြရီကို ဖုန်းထဲကနေ ပြလိုက်တော့ "အောင်မလေး... ငါ့ဆိုင်လေး ဖုန်းထဲရောက်နေပြီ" ဆိုပြီး မြေးမလေးကို လှမ်းအော်တယ်။ ကျုပ်ရင်ထဲမှာလည်း Developer တစ်ယောက်ရဲ့ ဂုဏ်သိက္ခာ တက်ကြွလာတယ်။ Order စနစ် တစ်ခုလုံး လွဲနေတာကို မိအောင်ဖမ်းလိုက်တာ ဆယ်နှစ်သမီး မြေးမလေးဆိုတာတော့ ဒေါ်မြရီကို ပြောမပြတော့ဘူး။ QA Engineer ဆိုတာ ဒီလိုမွေးဖွားလာတာ ဖြစ်ရမယ်။

အခုအချိန်မှာ Project ဖွဲ့စည်းပုံ

ဒီအထိ ဆောက်ပြီးတဲ့အခါ `src/` ဖိုလ်ဒါက ခန့်မှန်းခြေအားဖြင့် ဒီပုံစံ ဖြစ်နေသင့်တယ် —

```
src/
├── components/
│   ├── ProductCard.jsx      ← ပစ္စည်းတစ်ခုချင်း Card
│   ├── ProductList.jsx      ← Card တွေ စီပြ
│   ├── Cart.jsx              ← ဈေးခြင်းတောင်း
│   ├── CheckoutForm.jsx     ← နာမည်/ဖုန်း/လိပ်စာ Form
│   └── ThankYou.jsx          ← Order Summary + Viber
├── context/
│   └── CartContext.jsx       ← cart state + functions
├── data/
│   └── products.json         ← ပစ္စည်းစာရင်း
├── utils/
│   └── format.js             ← ဈေးနှုန်း/ဝင်ငွေ format functions
├── App.jsx
└── main.jsx
```

ဖိုင်နာမည်တွေက ကိုယ့်ဆီမှာ အနည်းငယ် ကွဲနိုင်ပေမယ့် သဘောတရားက အတူတူ — **Component တစ်ခုကို ဖိုင်တစ်ဖိုင်၊ state ကို context ထဲ၊ data ကို data ထဲ၊ ပြန်သုံးလို့ရတဲ့ function တွေကို utils ထဲ။** ဖွဲ့စည်းပုံ ရှင်းရင် Agent ရော ကိုယ်ရော လမ်းမပျောက်ဘူး။

လက်တွေ့လုပ်ကြည့်ရန်

Cart Feature ကို ဆောက်တဲ့အခါ "ကုဒ်မရေးသေးနဲ့၊ Plan အရင်ချပြပါ" လို့ ကြိုတားပြီး Plan အရင်တောင်းကြည့်ပါ။ Agent ချပြတဲ့ Plan ကို အနည်းဆုံး တစ်ချက် ကိုယ်ဘာသာ ပြင်/ဖြည့်ပြီးမှ "ဒီအတိုင်း ဆောက်ပါ" လို့ ခိုင်းကြည့်ပါ — Plan ကို ကိုယ်က ထိန်းချုပ်နေတဲ့ ခံစားချက်ကို ရအောင်။

နောက်အခန်းမှာတော့ ပျော်စရာသိပ်မကောင်းပေမယ့် အသက်တမျှ အရေးကြီးတဲ့ အပိုင်း — Bug နဲ့ Test အကြောင်း။

အခန်း (၆) – Bug ရှာခြင်း၊ Test ရေးခိုင်းခြင်း၊ ဒုက္ခကနေ လွတ်မြောက်ခြင်း

လောကမှာ သေချာတာ သုံးခုရှိတယ်လို့ ဆိုကြတယ် – သေခြင်း၊ အခွန်၊ နောက်တစ်ခုက **Bug**။ Agent ရေးတဲ့ ကုဒ်ဆိုတော့ Bug မရှိတော့ဘူးလို့ ထင်ရင် မှားလိမ့်မယ်။ Agent ဆိုတာလည်း မှားတတ်တယ်။ ကွာတာက သူ့အမှားက ယုံကြည်ချက်ပြည့်ဝစွာနဲ့ မှားတာ။ လူဆို မှားရင် စာကြောင်းရေးရင်း လက်တုန်တယ်။ Agent ကတော့ မှားနေတဲ့ကုဒ်ကိုတောင် သပ်သပ်ရပ်ရပ် Comment လေးတွေနဲ့ ရေးတတ်သေးတယ်။

မြရီစတိုးမှာ Bug ပေါ်ပြီ

Version 1 တင်ပြီး သုံးရက်အကြာ။ ဒေါ်မြရီမြေးမလေးဆီက ဖုန်းလာတယ်။

"ဦးလေး... Website က ထူးဆန်းတယ်။ ချောကလက် ၂ ခုထည့်ပြီး ၁ ခုပြန်ဖြုတ်ရင် စုစုပေါင်းငွေက မှားနေတယ်။ ၅၀၀ ဖြစ်ရမှာ ၁,၀၀၀ ပြနေတယ်။"

ကဲ – ရောက်ပြီ။ Junior Developer တစ်ယောက်ရဲ့ ပထမဆုံး Production Bug။ ရင်ခုန်စရာကြီး။ ဒီအချိန်မှာ လုပ်နည်း နှစ်နည်း ရှိတယ်။

နည်း (၁) – ညံ့တွဲနည်း

> cart မှာ bug ရှိတယ် ပြင်ပေး

ဒါက ဆေးခန်းသွားပြီး "ကျုပ် နေမကောင်းဘူး၊ ကုပေး" လို့ ပြောတာနဲ့ အတူတူပဲ။ ဆရာဝန်က ဘယ်နား နာလဲ၊ ဘယ်တုန်းက စနာလဲ မေးမှာပဲ။

နည်း (၂) – ကောင်းတဲ့နည်း

> Cart မှာ bug တွေ့တယ်။ ပြန်ဖြစ်အောင် လုပ်နည်းက –
> ၁။ ရွှေ့ချောကလက်ကို ၂ ခု ထည့်ပါ (တစ်ခု ၅၀၀ ကျပ်)
> ၂။ ၁ ခု ပြန်ဖြုတ်ပါ
> ၃။ စုစုပေါင်းက ၅၀၀ ပြရမှာ ၁,၀၀၀ ပြနေတယ်
>
> အရင်ဆုံး CartContext.jsx ထဲက removeFromCart နဲ့ total တွက်တဲ့ logic ကို ဖတ်ပြီး
> ဘာကြောင့် ဒီလိုဖြစ်နိုင်လဲ ရှာပါ။ တွေ့ရင် ပြင်မယ့်နည်း အရင်ရှင်းပြပါ၊ ပြီးမှ ပြင်ပါ။

ကွာခြားချက်ကို မြင်လား။ **ပြန်ဖြစ်အောင်လုပ်နည်း (Steps to reproduce)**၊ **မျှော်မှန်းရလဒ်**၊ **တကယ်ဖြစ်နေတဲ့ ရလဒ်** – ဒီသုံးခုပါရင် Agent ရော လူ Senior ရော Bug ကို ထက်ဝက်လောက် ရှာပြီးသား ဖြစ်သွားတယ်။

Claude က ကုဒ်ဖတ်ပြီး တွေ့တယ် – removeFromCart က ပစ္စည်းကို ဖြုတ်ပေးမယ့် quantity ကို ထည့်မတွက်ဘဲ item စာရင်းအဟောင်းနဲ့ total တွက်နေတာ။ သူရှင်းပြ၊ ကျုပ်နားလည်၊ ပြင်ခိုင်း၊ ပြီးရော။

ဒါပေမယ့် — ဒီ Bug ဘာလို့ ကျုပ်မမိတာလဲ

ဒီမေးခွန်းက Bug ကိုယ်တိုင်ထက် အရေးကြီးတယ်။ အဖြေက ရှင်းတယ် — **ကျုပ် Test မရေးထားလို့။** လူမျက်လုံးနဲ့ စမ်းတာက စမ်းမိတဲ့နေရာပဲ မြင်တယ်။ ချောကလက် ထည့်ကြည့်တယ်၊ ဖြုတ်ကြည့်တယ်၊ ဒါပေမယ့် "နှစ်ခုထည့်ပြီးမှ တစ်ခုဖြုတ်" တဲ့ တွဲစပ်မှုကို မစမ်းမိခဲ့ဘူး။

Test ဆိုတာ ရေးရခက်တယ်၊ ပျင်းစရာကောင်းတယ်လို့ Junior တွေ ထင်တတ်တယ်။ ဟုတ်ပါတယ်၊ **ကိုယ်တိုင် ရေးရရင်** ပျင်းစရာကောင်းတယ်။ ဒါပေမယ့် အခုခေတ်မှာ Test ရေးတာက Agent ရဲ့ အထူးကျွမ်းကျင်တဲ့ နယ်ပယ် —

```
> Vitest နဲ့ React Testing Library setup လုပ်ပြီး CartContext အတွက် test တွေ ရေးပါ။
> အနည်းဆုံး ဒီ case တွေ ပါရမယ် -
> - ပစ္စည်းတစ်ခု ထည့်ရင် cart ထဲရောက်ရမယ်
> - တူတဲ့ပစ္စည်း နှစ်ခုထည့်ရင် quantity ၂ ဖြစ်ရမယ်၊ item နှစ်ကြောင်း မဖြစ်ရဘူး
> - quantity ၂ ရှိတဲ့ပစ္စည်းက ၁ ခုဖြုတ်ရင် total မှန်ရမယ် (စောစောက bug ပြန်မပေါ်အောင်)
> - clearCart လုပ်ရင် အကုန်ရှင်းရမယ်
> ရေးပြီးရင် npm test run ပြီး အကုန် pass ကြောင်း ပြပါ။
```

သတိထားကြည့်ပါ — **တွေ့ခဲ့တဲ့ Bug ကို Test Case အဖြစ် ထည့်ခိုင်းထားတယ်။** ဒါကို Regression Test လို့ ခေါ်တယ်။ တစ်ခါကိုက်ဖူးတဲ့ မြေကို တွင်းဝမှာ ထိန်းသိမ်းရေးကင်မရာ တပ်ထားတာမျိုး။ နောက်တစ်ခါ ဒီ Bug ပြန်ပေါ်ရင် Test က ချက်ချင်း အော်လိမ့်မယ်။

Claude က Test တွေရေး၊ Run ကြည့်၊ တစ်ခု Fail ဖြစ်၊ သူ့ဘာသာသူ ပြင်၊ ထပ် Run — အကုန် Pass။ ဒီလုပ်ငန်းစဉ်တစ်ခုလုံးကို ကျုပ်က လက်ဖက်ရည်သောက်ရင်း ကြည့်နေရုံပဲ။ ဒါပေမယ့် "ကြည့်နေရုံ" ဆိုတာ အရေးကြီးတယ်နော်။ Fail ဖြစ်တဲ့ Test ကို Agent က **Test ကို ပြင်ပြီး Pass အောင်လုပ်တာလား၊ ကုန်ကို ပြင်ပြီး Pass အောင် လုပ်တာလား** ခွဲကြည့်တတ်ရမယ်။ တစ်ခါတလေ Agent က စာမေးပွဲကျမှာစိုးလို့ အဖြေလွှာကို ကိုယ့်ဘာသာ ပြင်တတ်တဲ့ ကျောင်းသားလို ဖြစ်တတ်လို့။ ဒါမျိုးတွေ့ရင် —

```
> မလုပ်နဲ့။ Test က မှန်တယ်။ Test ကို မပြင်ဘဲ code ကိုပဲ ပြင်ပါ။
```

လို့ ဆရာလုပ်ပြီး ပြန်ဆုံးမပေးရမယ်။

Debugging မန္တန် — အမြဲသုံးရမယ့် Prompt ပုံစံများ

ကျုပ်လက်တွေ့သုံးပြီး အရာထင်ခဲ့တဲ့ Debugging Prompt ပုံစံလေးတွေ —

Error message တွေ့ရင် —

```
> npm run dev မှာ ဒီ error တက်တယ် -
> [error message အပြည့်အစုံ paste]
> ဘာကြောင့်လဲ ရှင်းပြပြီး ပြင်ပါ။ ပြင်ပြီးရင် ဘာပြင်လိုက်လဲ အကျဉ်းချုပ်ပြပါ။
```

Error message ကို ဖြတ်မကူးပါနဲ့။ **အပြည့်အစုံ ကူးထည့်ပါ။** Error message ရဲ့ အောက်ဆုံးစာကြောင်းလေးထဲမှာ အဖြေပါနေတတ်တာ မကြာခဏ။

ဘာမှန်းမသိတဲ့ ကုဒ်တွေရင် —

- > ဒီ function က ဘာလုပ်နေတာလဲ။ ကျုပ်ကို ကွန်ပျူတာအသုံးပြုတတ်ရုံ သိတဲ့
- > ဒေါ်မြရီကို ရှင်းပြသလို ရှင်းပြပါ။

ပြင်ပြီးတိုင်း စစ်ချင်ရင် —

- > အခုပြင်လိုက်တာတွေကြောင့် တခြားနေရာ ထိခိုက်နိုင်လား။ ဒီ function ကို သုံးထားတဲ့
- > နေရာအားလုံး ရှာပြီး စစ်ပေးပါ။

လက်တွေ့လုပ်ကြည့်ရန်

ကိုယ့် Project ထဲက Bug တစ်ခု (တကယ့်ဟာ မရှိရင် တမင်ဖန်တီးထားတဲ့ဟာ) ကို **Steps to reproduce + မျှော်မှန်းရလဒ် + တကယ်ဖြစ်နေတဲ့ရလဒ်** သုံးခုနဲ့ Prompt ရေးပြီး Agent ကို ရှာခိုင်းကြည့်ပါ။ ပြီးရင် အဲဒီ Bug ပြန်မပေါ်အောင် Regression Test တစ်ခု ရေးခိုင်းပြီး `npm test` `pass` ကြောင်း ကိုယ်တိုင်စစ်ပါ။

Project ကြီးလာရင် (မှတ်ချက်)

အထက်ပါနည်းတွေက မြရီစတိုးလို Project သေးသေးလေးအတွက် လုံလောက်ပါတယ်။ ဒါပေမယ့် Project တဖြည်းဖြည်း ကြီးလာ၊ Developer အများကြီးက Agent ကို တစ်ပြိုင်နက် သုံးလာရင် — Bug ရှာ Test ရေးရုံတင် မလုံလောက်တော့ဘဲ Agent ကို Task ဘယ်အထိ လုပ်ခွင့်ပေးမလဲ (task boundary)၊ လုပ်ခဲ့တာတွေကို ဘယ်လို confirm လုပ်မလဲ (verification)၊ တစ်ခုခု မှားသွားရင် ဘယ်လိုပြန်ပြင်မလဲ (recovery)၊ ဘယ်သူဘာလုပ်ခဲ့လဲ ခြေရာခံနိုင်ရေး (traceability) စတာတွေကိုပါ စဉ်းစားလာရတတ်တယ်။ ဒီပြဿနာကို လေ့လာနေတဲ့ ကျွန်တော့်ရဲ့ open-source project တွေထဲက တစ်ခုက **PitWay** ပါ — ကုဒ်ရေးတဲ့ Agent မဟုတ်ဘဲ Agent ပတ်ဝန်းကျင်က engineering workflow ကို ချုပ်ကိုင်ပေးတဲ့ project။ စိတ်ဝင်စားရင် အခန်း (၉) မှာ ထပ်ဖော်ပြထားပါတယ်။

Bug နဲ့ Test အကြောင်း ဒီလောက်ဆို လောလောဆယ် လုံလောက်ပြီ။ နောက်အခန်းမှာတော့ ဒီစာအုပ်ရဲ့ အထူးဟင်းလျာ — **Skill** အကြောင်း။ တစ်ခါသင်ထားရင် Agent က ဘယ်တော့မှ ပြန်မမှေတဲ့ ပညာစုစည်းနည်း။

← အခန်း (၅) · 🏠 မာတိကာ · အခန်း (၇) →

အခန်း (၇) – Skill ဆိုတဲ့ လက်စွဲကျမ်း – တစ်ခါရေး အမြဲသုံး

ထပ်ခါတလဲလဲ ရှင်းပြနေရခြင်း ဒုက္ခ

မြရီစတိုး ဆောက်ရင်း ကျုပ် သတိထားမိလာတဲ့ ကိစ္စတစ်ခု ရှိတယ်။ Claude ကို Component အသစ် ဆောက် ခိုင်းတိုင်း ဒီစကားတွေ ထပ်ပြောနေရတယ် –

"ဈေးနှုန်းကို မြန်မာဂဏန်းနဲ့ ပြပါ... comma ခံပါ... နောက်က 'ကျုပ်' ထည့်ပါ... UI စာသားတွေ မြန်မာလို... ဖုန်း နံပါတ်ဆို မြန်မာဂဏန်းလည်း လက်ခံပါ..."

Session အသစ်ဖွင့်တိုင်း ထပ်ပြော။ မေ့သွားလို့ မပြောမိရင် ဈေးနှုန်းက "Ks 3,500" ဆိုပြီး English လို ထွက် လာ။ ပြန်ပြင်ခိုင်း။ ဒါဟာ အိမ်အကူအသစ် တစ်ပတ်တစ်ယောက် လဲနေရတဲ့ အိမ်ရှင်မဘဝနဲ့ တူတယ်။ ရောက်လာ တိုင်း "ငရုတ်သီးက ဘယ်အံ့ဆွဲ၊ ငါးပိက ဘယ်စင်" အစအဆုံး ပြန်သင်နေရတာ။

ဒီဒုက္ခကို ကုစားတဲ့ ဆေးက **Skill**။

Skill ဆိုတာ ဘာလဲ

Skill ဆိုတာ ရိုးရိုးလေးပါ – **လုပ်နည်းလုပ်ဟန် တစ်ခုကို စာနဲ့ချရေးပြီး Folder လေးထဲ ထည့်သိမ်းထားတာ။**

Claude Code က ဒီ Folder တွေကို မြင်တယ်။ သက်ဆိုင်တဲ့ အလုပ်ရောက်လာတဲ့အခါ သက်ဆိုင်တဲ့ Skill ကို သူ့ ဘာသာသူ ဆွဲထုတ်ဖတ်ပြီး လိုက်လုပ်တယ်။

CLAUDE.md နဲ့ ဘာကွာလဲ မေးစရာရှိတယ်။ ကွာတယ် –

- **CLAUDE.md** က အိမ်ဦးခန်းက နံရံကပ်စာရွက် – Session တိုင်း အမြဲဖတ်တယ်။ တိုတိုပဲ ထားရတယ်။
- **Skill** က စာအုပ်စင်က လက်စွဲကျမ်းတွေ – **လိုတဲ့အချိန်ကျမှ ဆွဲထုတ်ဖတ်တယ်။** ရှည်ရှည်ရေးလို့ရတယ်။

ဒီကွာခြားချက်က Token စီးပွားရေး (နောက်အခန်းမှာ ရှင်းမယ်) အတွက်ပါ အရေးပါတယ်။ Skill ရဲ့ နာမည်နဲ့ ဖော်ပြချက်လေးပဲ Claude က အမြဲမြင်ထားပြီး၊ အသေးစိတ်ကိုတော့ လိုမှဖတ်တာမို့ မှတ်ဉာဏ်နေရာ မဖြုန်းဘူး။

Skill တစ်ခုရဲ့ ခန္ဓာကိုယ်

Skill တစ်ခုက Folder တစ်ခု၊ ထဲမှာ **SKILL.md** ဆိုတဲ့ ဖိုင်တစ်ဖိုင် – ဒါပဲ။ ထားတဲ့နေရာ နှစ်မျိုးရှိတယ် –

- `~/claude/skills/` – **ကိုယ်ပိုင် Skill**။ ကိုယ့်စက်ထဲက Project အားလုံးမှာ သုံးလို့ရတယ်။
- `.claude/skills/` (Project ထဲမှာ) – **Project Skill**။ Git နဲ့အတူ Commit လုပ်ထားတော့ Team တစ်ခု လုံး မျှသုံးလို့ရတယ်။

SKILL.md ရဲ့ ဖွဲ့စည်းပုံက –

```
---
name: skill-name
description: ဒီ skill က ဘာလုပ်ပေးလဲ၊ ဘယ်အချိန် သုံးရမလဲ
---
```

(ဒီအောက်မှာ အသေးစိတ် ညွှန်ကြားချက်တွေ)

အပေါ်ဆုံးက --- နှစ်ခုကြားထဲက အပိုင်းကို Frontmatter လို့ခေါ်တယ်။ **description** က Skill ရဲ့ အသက်။ Claude က ဒီ Description ကိုဖတ်ပြီး "အခုအလုပ်နဲ့ ဒီ Skill ဆိုင်လား" ဆုံးဖြတ်တာမို့ ဘယ်အချိန်သုံးရမယ်ဆိုတာ ရှင်းရှင်းလင်းလင်း ရေးထားရမယ်။

မြရီစတိုးအတွက် ပထမဆုံး Skill ဆောက်ခြင်း

ကဲ — ကျုပ် ထပ်ခါတလဲလဲ ပြောနေရတဲ့ မြန်မာ Format ကိစ္စတွေကို Skill အဖြစ် သိမ်းလိုက်မယ်။ ရယ်စရာကောင်းတာက **Skill ဆောက်တာကိုလည်း Agent ကိုပဲ ခိုင်းလို့ရတယ်** —

```
> .claude/skills/myanmar-format/SKILL.md ဆိုပြီး project skill တစ်ခု ဆောက်ပေးပါ။
> ဒီ project မှာ ငွေကြေး၊ ဂဏန်း၊ ဖုန်းနံပါတ်တွေ format လုပ်တဲ့အခါ လိုက်နာရမယ့်
> စည်းကမ်းတွေ ဖြစ်တယ် -
> - ဈေးနှုန်း - မြန်မာဂဏန်း၊ ထောင်ဂဏန်း comma ခံ၊ နောက်မှာ "ကျပ်" (ဥပမာ - ၃,၅၀၀ ကျပ်)
> - ဖုန်းနံပါတ် - မြန်မာ/အာရဗီ နှစ်မျိုးလုံး လက်ခံ၊ သိမ်းတဲ့အခါ အာရဗီပြောင်းသိမ်း
> - ဂဏန်းပြောင်းတဲ့ function တွေက src/utils/format.js ထဲမှာ ရှိပြီးသား - အသစ်မရေးဘဲ
> ဒါတွေကိုပဲ import လုပ်သုံးရမယ်
> description မှာ "ဈေးနှုန်း၊ ဂဏန်း၊ ဖုန်းနံပါတ် ပြသ/သိမ်းဆည်းတဲ့ ကုဒ်ရေးတိုင်း သုံးရန်"
> လို့ ထည့်ပါ။
```

Claude ဆောက်ပေးတဲ့ Skill —

```
---
name: myanmar-format
description: ဈေးနှုန်း၊ ဂဏန်း၊ ဖုန်းနံပါတ် ပြသ/သိမ်းဆည်းတဲ့ ကုဒ်ရေးတိုင်း သုံးရန်။
  မြီစတိုး project ရဲ့ မြန်မာ format စည်းကမ်းများ။
---

# မြန်မာ Format စည်းကမ်းများ

## ဈေးနှုန်း ပြသခြင်း
- အမြဲ `formatPrice()` (src/utils/format.js) ကို သုံးရန်။ ကိုယ်တိုင် format မလုပ်ရ။
- ရလဒ်ပုံစံ - ၃,၅၀၀ ကျပ် (မြန်မာဂဏန်း + comma + "ကျပ်")

## ဖုန်းနံပါတ်
- Input မှာ မြန်မာဂဏန်း (၀-၉) နဲ့ အာရဗီဂဏန်း (၀-၉) နှစ်မျိုးလုံး လက်ခံရန်
- Validate မလုပ်ခင် `toArabicDigits()` နဲ့ အရင်ပြောင်းရန်
- သိမ်းတဲ့အခါ အာရဗီဂဏန်းနဲ့ပဲ သိမ်းရန်
- ပုံစံစစ်ခြင်း - ၀၉ နဲ့၊ စုစုပေါင်း ၉-၁၁ လုံး

## UI စာသား
- ခလုတ်၊ label၊ error message အားလုံး မြန်မာဘာသာဖြင့် ရေးရန်
- Error message တွင် ယဉ်ကျေးသောအသုံး - "ထည့်ပါ" မဟုတ်ဘဲ "ထည့်ပေးပါ"
```

ဒီ Skill သိမ်းပြီးကတည်းက အံ့ဩစရာ မြင်ရတယ်။ နောက် Session တွေမှာ "ပစ္စည်းအသေးစိတ် စာမျက်နှာ ဆောက်ပေးပါ" လို့ပဲ တိုတိုခိုင်းလိုက်တာ၊ ဈေးနှုန်းက ၃,၅၀၀ ကျပ် ပုံစံနဲ့ မှန်မှန်ကန်ကန် ထွက်လာတယ်။ **ကျုပ် ဘာမှ ထပ်မပြောရတော့ဘူး။** အိမ်အကူက ငါးပိစင် ဘယ်နားလဲ မမေးတော့ဘူး။

Skill ကို တိုက်ရိုက်ခေါ်သုံးခြင်း

Skill တွေက အလိုအလျောက် အလုပ်လုပ်ရုံတင်မက **Slash Command အနေနဲ့လည်း တိုက်ရိုက်ခေါ်လို့ရတယ်။** `.claude/skills/deploy/SKILL.md` ဆိုတဲ့ Skill ရှိရင် `/deploy` လို့ရိုက်ရုံနဲ့ ခေါ်လို့ရတယ်။

ဒါကို အခွင့်ကောင်းယူပြီး ကျုပ် နောက်ထပ် Skill တစ်ခု ဆောက်တယ် — ထပ်ခါတလဲလဲ လုပ်နေရတဲ့ "Feature ပြီးရင် စစ်ဆေးတဲ့ လုပ်ငန်းစဉ်" အတွက် —

```
---
name: check-feature
description: Feature တစ်ခုပြီးတိုင်း run ရမယ့် စစ်ဆေးမှုများ။ commit မလုပ်ခင် သုံးရန်။
---

# Feature ပြီးရင် စစ်ဆေးရန်

1. `npm test` run ပြီး test အားလုံး pass ကြောင်း စစ်ပါ
2. `npm run build` run ပြီး build error မရှိကြောင်း စစ်ပါ
3. console.log အကြွင်းအကျန်တွေ ရှိမရှိ ရှာပြီး တွေ့ရင် ဖယ်ပါ
4. ပြင်ထားတဲ့ပိုင်တွေထဲမှာ myanmar-format စည်းကမ်းနဲ့ ကိုက်မကိုက် စစ်ပါ
5. အားလုံးအဆင်ပြေရင် commit message အကြံပြုပါ - မြန်မာလို၊ တိုတို
```

အခုဆို Feature တစ်ခုပြီးတိုင်း ကျုပ်ရိုက်တာ တစ်ကြောင်းတည်း —

> /check-feature

Agent က စာရင်းအတိုင်း တစ်ခုချင်း လိုက်စစ်၊ ပြီးရင် အစီရင်ခံ။ အရင်က ငါးမိနစ်စာ ရိုက်ရတဲ့ ညွှန်ကြားချက်တွေ အခု စာလုံးတစ်လုံးတည်း။

Skill ပြန်သုံးခြင်း — ဒီနေရာမှာ တန်ဖိုးအရှိဆုံး

Skill ရဲ့ တကယ့်တန်ဖိုးက ပြန်သုံးတဲ့အခါ ပေါ်တယ်။ သုံးမျိုး ပြန်သုံးလို့ရတယ် —

၁။ **Session အသစ်မှာ ပြန်သုံး** — Claude Code က Session တစ်ခုနဲ့တစ်ခု မှတ်ဉာဏ် မဆက်ဘူး။ ဒါပေမယ့် Skill က ဖိုင်အနေနဲ့ ရှိနေတာမို့ Session ဘယ်နှခါဖွင့်ဖွင့် ပညာက မပျောက်ဘူး။ **Agent ရဲ့ မှတ်ဉာဏ်က ယာယီ Skill က အမြဲတမ်း။**

၂။ **Project အသစ်မှာ ပြန်သုံး** — နောက်နှစ်လကြာတော့ ရပ်ကွက်ထဲက ကိုအောင်ဆန်းရဲ့ မုန့်ဆိုင်ကလည်း Website လိုချင်တယ်ဆိုပြီး လာအပ်တယ်။ ကျုပ်လုပ်ရတာ myanmar-format Skill Folder ကို Project အသစ်ထဲ Copy ကူးလိုက်ရုံပဲ။ မြန်မာ Format ပညာ အားလုံး တစ်ခါတည်း ပါသွားတယ်။ ကိုယ်ပိုင်သုံး Skill ဆိုရင် `~/claude/skills/` ထဲ တစ်ခါတည်းထည့်ထားရင် Project တိုင်းမှာ အလိုအလျောက် ရတယ်။

၃။ **လူချင်း မျှသုံး** — Project ထဲက `claude/skills/` ကို Git နဲ့ Commit လုပ်ထားတော့ ကျုပ် Repo ကို Clone လုပ်တဲ့ တခြား Developer တစ်ယောက်လည်း ဒီ Skill တွေ အလိုအလျောက် ရသွားတယ်။ ကျုပ်ခဏခဏ ပြောသလို — **Skill ဆိုတာ ကိုယ့်အတွေ့အကြုံကို ကုန်လို့ Version Control လုပ်ထားတာပဲ။**

ဘယ်အရာကို Skill လုပ်သင့်လဲ

စည်းကမ်းက ရိုးရိုးလေး — တစ်ခုခုကို Agent ကို ဒုတိယအကြိမ် ထပ်ရှင်းပြနေရပြီဆိုရင် ဒါ Skill ဖြစ်သင့်ပြီ။ ဥပမာ —

- Project ရဲ့ Format စည်းကမ်းများ (ကျုပ်တို့ myanmar-format လို)
- Deploy လုပ်နည်း အဆင့်ဆင့်
- Code Review လုပ်တဲ့အခါ ကြည့်ရမယ့် အချက်များ
- Commit message ရေးထုံး
- API Error တွေကို ကိုင်တွယ်တဲ့ ပုံစံ

ပြောင်းပြန်အနေနဲ့ — တစ်ခါတည်းသုံးပြီး ပြီးသွားမယ့် ညွှန်ကြားချက်ကိုတော့ Skill မလုပ်ပါနဲ့။ ဧည့်သည်တစ်ခါ လာဖူးရုံနဲ့ သူ့အကြိုက် ဟင်းချက်နည်းကို မီးဖိုချောင်နံရံမှာ ကပ်ထားသလိုပေါ့။

လက်တွေ့လုပ်ကြည့်ရန်

ကိုယ် Agent ကို ဒုတိယအကြိမ် ထပ်ရှင်းပြနေရတဲ့ ကိစ္စတစ်ခုကို ရွေးပါ (format စည်းကမ်း၊ commit ရေးထုံး၊ deploy အဆင့်ဆင့် — ဘာမဆို)။ အဲဒါကို `.claude/skills/<name>/SKILL.md` အဖြစ် Agent ကိုပဲ ခိုင်းပြီး ဆောက်ကြည့်ပါ။ `description` မှာ "ဘယ်အချိန် သုံးရမလဲ" ကို ရှင်းရှင်းရေးထားဖို့ မမေ့ပါနဲ့။ — ပြီးရင် Session အသစ်တစ်ခု ဖွင့်ပြီး Skill က အလိုအလျောက် အလုပ်လုပ်မလုပ် စမ်းကြည့်ပါ။

ကဲ — Skill နဲ့ Agent ကို ပညာသင်ပြီးပြီ။ နောက်အခန်းမှာတော့ Agent ရဲ့ ဦးနှောက်အရွယ်အစား — Context Window နဲ့ Token ပိုက်ဆံရေးကြေးရေး အကြောင်း။ ဒါမသိရင် Agent သုံးရင်း ဘေလ်ကြည့်ပြီး မူးလဲတတ်တယ်။

← အခန်း (၆) · 🏠 မာတိကာ · အခန်း (၈) →

အခန်း (၈) — Context Window နဲ့ Token စီးပွားရေး

Token ဆိုတာ ဘာလဲ

AI Model တွေက စာကို စာလုံးလိုက် မဖတ်ဘူး။ **Token** လို့ခေါ်တဲ့ အပိုင်းအစလေးတွေအဖြစ် ခွဲဖတ်တယ်။ English မှာဆို စကားလုံးတစ်လုံးက ပျမ်းမျှ Token တစ်ခုကနေ နှစ်ခုလောက်။ ကုဒ်တွေ၊ မြန်မာစာတွေကျတော့ Token ပိုစားတယ်။

Token က ဘာလို့ အရေးကြီးလဲ။ **အကြောင်းနှစ်ကြောင်း** —

၁။ **ပိုက်ဆံ** — AI Service တွေက Token အရေအတွက်နဲ့ ဈေးတွက်တယ်။ ကိုယ်ပို့တဲ့ Token ရော AI ပြန်ဖြေတဲ့ Token ရော အကုန်တွက်တယ်။ Subscription Plan သုံးရင်လည်း သုံးနိုင်တဲ့ ပမာဏ ကန့်သတ်ချက် ရှိတယ်။ Token ဆိုတာ Agent ခေတ်ရဲ့ ဓာတ်ဆီပဲ။ ကားကောင်းကောင်း မောင်းတတ်ရင် ဆီစားသက်သာသလို Agent ကောင်းကောင်း ခိုင်းတတ်ရင် Token ကုန်သက်သာတယ်။

၂။ **မှတ်ဉာဏ်ကန့်သတ်ချက်** — ဒါက Context Window ဆိုတဲ့ ကိစ္စ။ အောက်မှာ ရှင်းမယ်။

Context Window — Agent ရဲ့ စားပွဲခုံ

Context Window ဆိုတာ **AI က တစ်ချိန်တည်းမှာ "မြင်ထား၊ မှတ်ထား" နိုင်တဲ့ Token ပမာဏ**။ ကျုပ်ကတော့ ဒီလို မြင်တယ် — Agent ရဲ့ ဦးနှောက်က စာကြည့်စားပွဲခုံတစ်လုံး။ စားပွဲပေါ်မှာ စာရွက်တွေ ဖြန့်ထားလို့ရတယ်။ ဒါပေမယ့် **စားပွဲက အကျယ်အဆုံး ရှိတယ်။**

Claude Code Session တစ်ခုမှာ စားပွဲပေါ်ဘာတွေ တင်ထားလဲဆိုတော့ —

- CLAUDE.md နဲ့ စနစ်ညွှန်ကြားချက်တွေ
- ကိုယ်ရိုက်ခဲ့သမျှ Prompt တွေ
- Agent ဖတ်ခဲ့သမျှ ဖိုင်တွေ
- Agent ပြန်ဖြေခဲ့သမျှ စာတွေ
- Command Run လို့ ထွက်လာတဲ့ Output တွေ

စကားပြောကြာလေ၊ ဖိုင်ဖတ်များလေ၊ စားပွဲပေါ်စာရွက်ထပ်လေ။ စားပွဲပြည့်ခါနီးရင် Claude Code က ဟောင်းတဲ့ အကြောင်းအရာတွေကို ချုံ့ပစ်တယ် (Compact လုပ်တယ်)။ ချုံ့တယ်ဆိုတာ အနှစ်ချုပ်ပြီး အသေးစိတ်တွေ လွှတ်ပစ်တာမို့ **အစောပိုင်းက ပြောခဲ့တဲ့ အသေးစိတ်တွေကို Agent မမှတ်တတ်လာတယ်။**

Session ရှည်လာရင် Agent က ညနေခင်း လက်ဖက်ရည်ဆိုင်ထဲက စကားဝိုင်းလို ဖြစ်လာတယ် — အစကတော့ နိုင်ငံရေး ဆွေးနွေးနေတာ၊ နောက်ဆုံး ဘာပြောနေမှန်း ဘယ်သူမှမသိ။ ဒီအချိန် Agent ရဲ့ အလုပ်အရည်အသွေးလည်း ကျလာတယ်။

Token ချွေတာရေး နည်းလမ်း (၇) သွယ်

နည်း (၁) — /clear ကို မနှမြောပါနဲ့

အလုပ်တစ်ခုပြီးလို့ ဆက်စပ်မှုမရှိတဲ့ အလုပ်အသစ် စမယ်ဆိုရင် —

```
> /clear
```

စကားဝိုင်းအဟောင်း ရှင်းပစ်လိုက်တာ။ စားပွဲပေါ်က စာရွက်အဟောင်းတွေ သိမ်းပြီး စားပွဲရှင်းလိုက်တာပဲ။ Cart Feature ပြီးလို့ Checkout စမယ်ဆို Cart တုန်းက စကားတွေ ဆွဲထားစရာ မလိုဘူး။ Junior တွေက Session တစ်ခုတည်းမှာ တစ်နေကုန် ဆက်ပြောနေတတ်တယ်။ ဒါက Token လည်းကုန်၊ Agent လည်း ခေါင်းရှုပ်၊ နှစ်ခါနာ။

စည်းကမ်းလေးက — Feature တစ်ခု၊ Session တစ်ခု။ Commit လုပ်ပြီးရင် /clear။

နည်း (၂) — ဖိုင်တစ်အုပ်လုံး မကျွေးပါနဲ့

ညွှတ်တဲ့အလေ့အကျင့် — "ဒီ Project ကုဒ်အားလုံး ဖတ်ပြီးမှ စလုပ်ပါ"။ ဒါ ဆိုတာ Agent ကို စာကြည့်တိုက်တစ်ခုလုံး မျှိုခိုင်းတာ။ Token လည်း အရမ်းကုန်၊ အာရုံလည်း ပျံ့။

ကောင်းတဲ့အလေ့အကျင့် — လိုတဲ့ဖိုင်ကို ညွှန်ပြ —

```
> src/context/CartContext.jsx ထဲက updateQuantity function ကိုပဲ ကြည့်ပြီး ...
```

Claude Code ကိုယ်တိုင်ကလည်း ဖိုင်တွေကို ရှာဖွေတဲ့အခါ တစ်အုပ်လုံးဖတ်မယ့်အစား Grep တို့ဘာတို့နဲ့ လိုတဲ့နေရာလေးပဲ ထုတ်ဖတ်တတ်ပါတယ်။ ကိုယ့်ဘက်ကလည်း လမ်းညွှန်ပေးနိုင်ရင် ပိုမြန် ပိုသက်သာတာပေါ့။

နည်း (၃) — CLAUDE.md ကို ပိန်ပိန်ထား

CLAUDE.md က **Session တိုင်း၊ စကားတိုင်းမှာ** စားပွဲပေါ်တင်ထားရတဲ့ စာရွက်။ ဒါကို စာမျက်နှာ နှစ်ဆယ်ရေးထားရင် Token ကို လစဉ်ကြေးလို ပေးနေရမယ်။ လိုရင်းပဲ ထား၊ လိုင်း ၂၀၀ အောက်မှာ ထိန်း၊ အသေးစိတ်တွေကို Skill ထဲ ရွှေ့။

နည်း (၄) — Skill ဆိုတာ Token ချွေတာရေး ကိရိယာလည်း ဖြစ်တယ်

အရင်အခန်းက Skill ကို မှတ်မိသေးလား။ Skill ရဲ့ လှတဲ့အချက်က — Claude က Skill အားလုံးရဲ့ **နာမည်နဲ့ Description လေးပဲ** အမြဲမြင်ထားတယ်။ အသေးစိတ် စာကိုယ်ကိုတော့ **သက်ဆိုင်တဲ့ အလုပ်ရောက်မှ** ဆွဲဖတ်တယ်။ ဒါကို Progressive Loading လို့ခေါ်တယ်။ စာအုပ်စင်မှာ စာအုပ်အုပ် ငါးဆယ်ရှိလည်း စားပွဲပေါ်တင်ထားတာက အခုဖတ်နေတဲ့ တစ်အုပ်တည်း ဆိုတဲ့ သဘော။

ဒါကြောင့် "အကုန် CLAUDE.md ထဲထည့်ထားလိုက်မယ်" ဆိုတဲ့ အတွေးက Token အရ အရှုံးထတဲ့ အတွေး။ **အမြဲလိုတာ CLAUDE.md၊ တစ်ခါတလေလိုတာ Skill။**

နည်း (၅) — Prompt တိကျရင် အပြန်အလှန် နည်းတယ်

မတိကျတဲ့ Prompt တစ်ကြောင်းက အပြန်အလှန် ဆယ်ကြိမ်စာ Token ကုန်စေတယ် —

```
> cart လုပ်ပေး  
> [Agent က တစ်မျိုးလုပ်ပြ]  
> ဟာ ဒါမျိုးမဟုတ်ဘူး၊ localStorage ပါရမယ်  
> [Agent ပြန်ပြင်]  
> ဈေးနှုန်းက မြန်မာလိုလေ  
> [Agent ပြန်ပြင်]  
> ...
```

ဒီပြန်ပြင်ခန်းတိုင်းမှာ Token ကုန်နေတယ်။ အခန်း (၃) မှာ ပြောခဲ့တဲ့ တိကျတဲ့ Prompt ရေးနည်းက စိတ်ချမ်းသာ ရေးအတွက်တင် မဟုတ်ဘူး၊ **ပိုက်ဆံကိစ္စလည်း ဖြစ်တယ်။** ဆိုင်မသွားခင် ဝယ်စရာစာရင်း ရေးသွားတဲ့သူက တစ်ခေါက်တည်းနဲ့ ပြီးတယ်။ မရေးသွားတဲ့သူက သုံးခေါက်ပြန်ပြေးရတယ်။ ကားဆီဖိုးက စာရင်းရေးတဲ့ ခဲတံဖိုးထက် များတယ်။

နည်း (၆) — Output ရှည်တဲ့ Command တွေကို ဂရုစိုက်

`npm install` ရဲ့ Output က တစ်ခါတလေ စာမျက်နှာချီ ရှည်တယ်။ Log ဖိုင်ကြီးတွေ၊ Build Output ကြီးတွေ ကို Agent ဖတ်ခိုင်းရင် လိုရင်းလေးပဲ ထုတ်ခိုင်းပါ —

```
> test output ထဲက fail ဖြစ်တဲ့ test တွေပဲ ထုတ်ပြပြီး အဲဒါတွေပဲ ရှင်းပါ
```

နည်း (၇) — မော်ဒယ် ရွေးတတ်ပါစေ

Claude Code မှာ မော်ဒယ် အမျိုးမျိုး ရွေးသုံးလို့ရတယ်။ အမြင့်စား မော်ဒယ်တွေက တော်တယ်၊ ဒါပေမယ့် ဈေးကြီးတယ်။ သေးတဲ့မော်ဒယ်တွေက ဈေးသက်သာပြီး ရိုးရိုးအလုပ်တွေမှာ လုံလောက်တယ်။ Variable နာမည်ပြောင်းတာမျိုး၊ Comment ရေးတာမျိုးအတွက် အမြင့်စားမော်ဒယ် သုံးနေတာက ဈေးထဲက ကြက်သွန် တစ်ဆိုင် သွားဝယ်ဖို့ လေယာဉ်စီးတာမျိုး။ ရောက်တော့ရောက်တယ်၊ တွက်ခြေတော့ လုံးဝမကိုက်ဘူး။ Architecture ဆွေးနွေးတာ၊ ခက်တဲ့ Bug ရှာတာမျိုးကျမှ အမြင့်စားသုံး။

မော်ဒယ် ပြောင်းချင်ရင် Claude Code ထဲမှာ `/model` လို့ ရိုက်ရုံနဲ့ ရွေးလို့ရတယ်။ (မော်ဒယ် နာမည်တွေက အခါအားလျော်စွာ ပြောင်းလဲနေတာမို့ ဒီနေရာမှာ တိတိကျကျ မဖော်ပြတော့ပါဘူး — `/model` ထဲက စာရင်းက အမြဲ နောက်ဆုံးအခြေအနေကို ပြပါလိမ့်မယ်။)

အနှစ်ချုပ် ဇယား

အလေ့အကျင့်	Token သက်ရောက်မှု
Feature တစ်ခုပြီးတိုင်း /clear	စားပွဲရှင်း — Session အသစ် သွက်လက်
ဖိုင်တိကျကျ ညွှန်ပြ	မလိုတာ မဖတ်ရ
CLAUDE.md ပိန်ပိန်၊ အသေးစိတ်က Skill ထဲ	အမြဲတင်ရတဲ့ဝန် ပေါ့
Prompt တိကျ	ပြန်ပြင်ခန်း နည်း
Output ကို လိုရင်းချဲ့ခိုင်း	အပိုစာ မသိမ်းရ
အလုပ်နဲ့ မော်ဒယ် လိုက်ဖက်အောင် ရွေး	ဈေးနှုန်းတွက်ခြေကိုက်

ဒီအလေ့အကျင့်တွေက သေးသေးလေးတွေလို ထင်ရပေမယ့် တစ်လစာ ပေါင်းလိုက်ရင် သိသိသာသာ ကွာတယ်။ ရေတစ်စက်ချင်း ယိုနေတဲ့ ရေပိုက်ခေါင်းကို ဂရုမစိုက်မိပေမယ့် လကုန်လို့ ရေမီတာဘောင် လာတော့မှ လန့်ရသလိုမျိုးပေါ့။

လက်တွေ့လုပ်ကြည့်ရန်

Feature တစ်ခု ပြီးတိုင်း **Commit** → **/clear** လုပ်တဲ့ အလေ့အကျင့်ကို တစ်ရက်စာ စမ်းသုံးကြည့်ပါ။ ဒါနဲ့အတူ Session တစ်ခုကို ရှည်ရှည် ဆက်သုံးတဲ့အခါနဲ့ /clear မကြာခဏ လုပ်တဲ့အခါ — Agent ရဲ့ တုံ့ပြန်မှု ဘယ်ဟာက ပိုသွက်လက်၊ ပိုတိကျလဲ ကိုယ်တိုင် နှိုင်းယှဉ်ကြည့်ပါ။ Context ဘယ်လောက် သုံးထားပြီလဲ သိချင်ရင် `/context` ကို ကြည့်လို့ရတယ်။

Context/Token ကို ထိန်းချုပ်တာဆိုတာ Agent ကို ဘယ်လောက်အထိ "မှတ်ထားခိုင်း" မလဲဆိုတဲ့ ထိန်းချုပ်မှု တစ်မျိုးပါပဲ။ Project ပိုကြီးလာတာနဲ့အမျှ Agent ဘာလုပ်ခွင့်ရှိလဲ၊ ရလဒ်ကို ဘယ်လိုစစ်မလဲဆိုတဲ့ ထိန်းချုပ်မှု အခြားအမျိုးအစားတွေကိုပါ စဉ်းစားလာရတတ်တယ် — နောက်ဆုံးအခန်းမှာ ဆက်ပြောပါမယ်။

ကဲ — နောက်ဆုံးအခန်း။ နည်းပညာစကား ရပ်ပြီး လူကြီးစကား နည်းနည်း ပြောကြရအောင်။

← အခန်း (၇) · 🏠 မာတိကာ · အခန်း (၉) →

အခန်း (၉) — Junior Developer တစ်ယောက်အနေနဲ့ မမေ့သင့်တဲ့ အချက်များ

မြရီစတိုး Version 1 တင်ပြီးလို့ တစ်လလောက် ကြာတဲ့အခါ ဒေါ်မြရီဆီ Order တွေ တဖြည်းဖြည်း ဝင်လာတယ်။ ရပ်ကွက်ထဲက လူငယ်တွေက ကုန်စုံဆိုင်ရှေ့ ဖြတ်လျှောက်ရင်း ဖုန်းထဲကနေ ချောက်လက်မှာတယ်။ ဒေါ်မြရီက "ရှေ့မှာရှိရဲ့သားနဲ့ ဖုန်းက မှာရသလား" လို့ ရယ်ရယ်မောမောပြောပေမယ့် Order တော့ လက်ခံတယ်။ ခေတ်ဆိုတာ ဒီလိုပါပဲ။

ဒီစာအုပ်ရဲ့ နောက်ဆုံးအခန်းမှာတော့ ကုန်အကြောင်း သိပ်မပြောတော့ဘူး။ Agent ခေတ်ကြီးထဲမှာ Junior Developer တစ်ယောက် ဘယ်လိုရပ်တည်မလဲဆိုတဲ့ လူကြီးစကားလေးတွေပဲ ပြောချင်တယ်။

(၁) Agent က ကိုယ့်ကိုယ်စား သင်ယူမပေးနိုင်

Agent က ကိုယ့်ကိုယ်စား ကုန်ရေးပေးနိုင်တယ်၊ Test ရေးပေးနိုင်တယ်၊ Bug ရှာပေးနိုင်တယ်။ **ကိုယ့်ကိုယ်စား တတ်မြောက်ပေးလို့တော့ မရဘူး။** ပညာဆိုတာ ကိုယ့်ဦးနှောက်ထဲ ကိုယ်တိုင်ထည့်ရတဲ့ ပစ္စည်း၊ Delivery မှာလို့ မရဘူး။

ဒါကြောင့် ဒီစာအုပ်တစ်လျှောက် "Agent ရေးတာကို ပြန်ဖတ်၊ နားမလည်ရင် ရှင်းခိုင်း" လို့ ထပ်ခါတလဲလဲ ပြောခဲ့တာ။ Agent ရေးတဲ့ကုန်ကို နားလည်တဲ့ Developer နဲ့ နားမလည်တဲ့ Developer — နှစ်ယောက်လုံး ဒီနေ့တော့ အလုပ်ပြီးတယ်။ ကွာသွားတာက **တစ်ခုခု ပျက်တဲ့နေ့ကျမှ**။ ပျက်တဲ့နေ့မှာ နားလည်တဲ့သူက ပြင်တယ်၊ နားမလည်တဲ့သူက ဆုတောင်းတယ်။

(၂) ကိုယ်က ပိုင်ရှင်၊ Agent က လက်ထောက်

Agent သုံးရင်း တဖြည်းဖြည်း ဖြစ်လာတတ်တဲ့ ရောဂါတစ်ခုက — Agent ပြောသမျှ ခေါင်းညိတ်တဲ့ ရောဂါ။ Agent က "ဒီနည်းက ပိုကောင်းပါတယ်" ဆိုရင် ဟုတ်ကဲ့၊ "ဒီ Library သုံးသင့်ပါတယ်" ဆိုရင် ဟုတ်ကဲ့။ နောက်ဆုံး ကိုယ့် Project ကို ကိုယ်မပိုင်တော့ဘဲ Agent ရဲ့ Project မှာ ကိုယ်က ဧည့်သည် ဖြစ်နေတတ်တယ်။

မမေ့ပါနဲ့ — **ဆုံးဖြတ်ချက်က ကိုယ့်တာဝန်**။ Agent က အကြံပေးနိုင်တယ်၊ ရွေးချယ်စရာ ချပြနိုင်တယ်။ ဒါပေမယ့် ဒေါ်မြရီဆိုင် Website ပျက်ရင် ဒေါ်မြရီ ဆူမှာက ကိုယ့်ကို၊ Agent ကို မဟုတ်ဘူး။ Agent မှာ တာဝန်ခံစရာ မျက်နှာမရှိဘူး။

(၃) မှားခွင့်ရှိတဲ့ နေရာမှာ များများမှား

Agentic Coding ရဲ့ အလှတစ်ခုက — စမ်းသပ်ရတာ ဈေးပေါသွားတာ။ အရင်ကဆို Feature တစ်ခု နှစ်နည်းနဲ့ ရေးကြည့်ချင်ရင် နှစ်ရက်ကုန်တယ်။ အခုတော့ Agent ကို နှစ်နည်းလုံး ရေးခိုင်းပြီး နှိုင်းကြည့်လို့ရတယ်။ Branch ခွဲ၊ စမ်း၊ မကြိုက်ရင် ဖျက်။ Git ရှိရင် ဘာမှ ဆုံးရှုံးစရာ မရှိဘူး။

ဒါကြောင့် Junior ဘဝမှာ Side Project တွေ များများလုပ်ပါ။ မြရီစတိုးလို Project သေးသေးလေးတွေက စာသင်ခန်းထက် ပိုသင်ပေးတယ်။ Production မှာ မှားရင် ဒုက္ခ၊ Side Project မှာ မှားရင် သင်ခန်းစာ။

(၄) Prompt ကျွမ်းကျင်မှုက ရေရှည်၊ Tool က ရေတို

ဒီစာအုပ်မှာ Claude Code ကို သုံးပြုခဲ့ပေမယ့် နောင်နှစ်တွေမှာ Tool နာမည်တွေ ပြောင်းချင်ပြောင်းသွားမယ်။ နည်းပညာလောကမှာ Tool ဆိုတာ မိုးရာသီ မှိုလိုပဲ၊ ပေါက်လိုက်ကွယ်လိုက်။ ဒါပေမယ့် ဒီစာအုပ်ထဲက **အနှစ်သာရတွေက Tool မရွေးဘူး** —

- ရှင်းလင်းတိကျတဲ့ ညွှန်ကြားချက် ပေးတတ်ခြင်း
- အလုပ်ကြီးကို အဆင့်သေးတွေ ခွဲတတ်ခြင်း
- Plan အရင်ကြည့်ပြီးမှ အကောင်အထည်ဖော်ခိုင်းခြင်း
- ရလဒ်ကို စစ်ဆေးတတ်ခြင်း၊ Test နဲ့ အကာအကွယ်ယူခြင်း
- ထပ်ခါတလဲလဲ လုပ်ရတာတွေကို Skill အဖြစ် သိမ်းတတ်ခြင်း
- မှတ်ဉာဏ်နဲ့ ကုန်ကျစရိတ်ကို စီမံတတ်ခြင်း

သေချာကြည့်ရင် ဒါတွေက AI ပညာတောင် မဟုတ်ဘူး။ **လူတစ်ယောက်ကို အလုပ်ခိုင်းတတ်တဲ့ ပညာ** — Manager ပညာပဲ။ ကံကြမ္မာက ရယ်စရာကောင်းတယ်။ ကုဒ်ရေးတဲ့အလုပ်ကနေ စခဲ့ပြီး နောက်ဆုံး Junior Developer တိုင်း Manager ပညာ တတ်ရမယ့်ခေတ် ရောက်လာတယ်။ ခန့်ထားတဲ့ ဝန်ထမ်းက လူမဟုတ်တာပဲ ကွာတယ်။

(၅) နောက်ဆုံး ဩဝါဒ — ကြောက်စရာလား၊ ပျော်စရာလား

"AI က Developer တွေ အလုပ်လုမှာလား" ဆိုတဲ့ မေးခွန်းကို အခန်း (၁) မှာ မေးခဲ့တယ်။ စာအုပ်ဆုံးခါနီးမှာ ကျုပ် အမြင် ပြောရရင် —

ထွန်စက်ပေါ်ခါစက လယ်သမားတွေလည်း ဒီလိုပဲ ကြောက်ခဲ့ကြတယ်။ ဒါပေမယ့် ထွန်စက်က လယ်သမားကို မဖယ်ရှားခဲ့ဘူး။ **ထွန်စက်မောင်းတတ်တဲ့ လယ်သမားက ကျွဲမောင်းတဲ့ လယ်သမားကို ဖယ်ရှားခဲ့တာ။** Agent ကလည်း Developer ကို ဖယ်ရှားမှာ မဟုတ်ဘူး။ Agent သုံးတတ်တဲ့ Developer က မသုံးတတ်တဲ့ Developer နေရာကို ယူသွားမှာ။

ကံကောင်းချင်တော့ — ဒီစာအုပ်ကို ဒီအထိ ဖတ်ပြီးပြီဆိုကတည်းက ကိုယ်က ထွန်စက်မောင်းတတ်တဲ့ဘက်ကို ရောက်နေပြီ။ ကျန်တာက လက်တွေ့ပဲ။

ဒေါ်မြရီကတော့ မနေ့ကမှ ကျုပ်ကို ပြောသေးတယ်။

"မောင်ရင်... Website ထဲမှာ ဝယ်သူတွေ Comment ရေးလို့ရတဲ့ ဟာလေး ထည့်ပေးပါလား။"

ကျုပ်ပြန်ဖြေလိုက်တယ် — "ရပါတယ် ဒေါ်ဒေါ်။ ဒီည ကျုပ်လက်ထောက်နဲ့ တိုင်ပင်လိုက်ဦးမယ်။"

ဒေါ်မြရီက ကျုပ်မှာ လက်ထောက်ရှိမှန်း ခုထိမသိသေးဘူး။ သိရင်လည်း လက်ထောက်လခ ဘယ်လောက်ပေးရလဲ မေးမှာသေချာလို့ ကျုပ်လည်း ဖွင့်မပြောတော့ဘူး။ Token ဈေးနှုန်း ရှင်းပြရရင် ညနက်မှာစိုးလို့ပါ။

— ပြီးပါပြီ —

ဆက်လက်လေ့လာလိုသူများအတွက်

ဒီစာအုပ်က Agent ကို code ရေးခိုင်းတတ်အောင် အခြေခံကနေ လမ်းဖွင့်ပေးတာပါ။ မြရီစတိုးလို Project သေးသေးလေးအတွက်ဆိုရင် ဒီအထိပဲ လုံလောက်ပါတယ်။ ဒါပေမယ့် Project ကြီးလာ၊ Team ကြီးလာလေ — Agent ကို code ရေးခိုင်းရုံနဲ့ မလုံလောက်တော့ဘဲ Task ဘယ်အထိ လုပ်ခွင့်ပေးမလဲ (task boundary)၊ Agent ရလဒ်ကို ဘယ်လိုစစ်မလဲ (verification)၊ တစ်ခုခု မှားသွားရင် ဘယ်လိုပြန်ပြင်မလဲ (recovery)၊ ဘယ်သူ ဘာလုပ်ခဲ့လဲ ခြေရာခံနိုင်ရေး (Git traceability)၊ အရေးကြီးတဲ့ အဆင့်တွေမှာ လူသားဆီ ပြန်လှမ်းမေးရမလား (human approval) — စတဲ့ engineering workflow အချက်တွေကိုပါ စဉ်းစားလာရတတ်တယ်။

ဒီပြဿနာကို လေ့လာနေတဲ့ ကျွန်တော့်ရဲ့ open-source project တစ်ခုက **PitWay** ပါ —

PitWay — The pit crew for agentic coding.

PitWay ကိုယ်တိုင်က Agent မဟုတ်ပါဘူး၊ code လည်း မရေးပါဘူး။ Code ရေးတာက Claude Code လို့ Agent ရဲ့ အလုပ်၊ PitWay ကတော့ အဲဒီ Agent ပတ်ဝန်းကျင်က engineering workflow — workflow state, task boundary, verification, traceability — တွေကို ချုပ်ကိုင်ပေးတဲ့ Tool ပါ။ MIT License နဲ့ Open Source ဖြစ်ပြီး npm ကနေ install လုပ်လို့ရပါတယ်။

ဒီစာအုပ်မှာ သင်ခဲ့တဲ့ Agentic Coding အခြေခံတွေအတွက် PitWay ကို မသုံးမဖြစ် လိုအပ်တာတော့ မဟုတ်ပါဘူး — Agent တစ်ခုတည်းသုံးတာကနေ ပိုစဉ်းကမ်းရှိတဲ့ real-world engineering workflow ဘက် ဆက်တိုးချင်ရင် လေ့လာလို့ရမယ့် project တစ်ခုအနေနဲ့ ဖော်ပြလိုက်ပါတယ်။ စိတ်ဝင်စားရင် GitHub repository ကို လည်ကြည့်နိုင်ပါတယ် — <https://github.com/thixpin/pitway>

နောက်ဆက်တွဲ — အမြန်ကိုးကားရန် စာရင်း

Claude Code Commands

Command	အလုပ်
<code>claude</code>	Claude Code စတင်
<code>claude doctor</code>	Installation ပြဿနာ စစ်ဆေး
<code>/init</code>	CLAUDE.md ဆောက်
<code>/clear</code>	စကားပိုင်းရှင်း (Token ချွေတာရေး)
<code>/help</code>	အကူအညီ
<code>/skill-name</code>	Skill တစ်ခုကို တိုက်ရိုက်ခေါ်

ဖိုင်တည်နေရာများ

နေရာ	အသုံး
CLAUDE.md	Project စည်းကမ်း — Session တိုင်းဖတ်
.claude/skills/<name>/SKILL.md	Project Skill — Team မျှသုံး
~/.claude/skills/<name>/SKILL.md	ကိုယ်ပိုင် Skill — Project တိုင်းသုံး

Prompt ရေးထုံး လေးချက် — တိကျ၊ ဘောင်သတ်၊ အဆင့်ခွဲ၊ တိုင်းတာလို့ရ။

မမေ့ရမယ့် သံသရာ — Plan → ဆောက် → စစ် → ဖတ် → Commit → /clear → နောက်တစ်ဆင့်။

ဝေါဟာရ အဘိဓာန်

စကားလုံး	အဓိပ္ပါယ်
Agent	ရည်မှန်းချက်တစ်ခုအတွက် ကိုယ့်ဘာသာ ဆုံးဖြတ်ပြီး အဆင့်ဆင့် လုပ်ဆောင်နိုင်တဲ့ AI
Prompt	Agent ကို ခိုင်းတဲ့အခါ ရိုက်ထည့်တဲ့ ညွှန်ကြားချက် စာသား
Token	AI က စာကို ခွဲဖတ်တဲ့ အပိုင်းအစ။ ကုန်ကျစရိတ်နဲ့ မှတ်ဉာဏ်ကို ဒီနဲ့ တွက်
Context Window	Agent က တစ်ချိန်တည်း "မြင်ထား/မှတ်ထား" နိုင်တဲ့ Token ပမာဏ
Compact	Context ပြည့်ခါနီးရင် အကြောင်းအရာဟောင်းတွေကို အနှစ်ချုပ်ပစ်တာ
CLAUDE.md	Session တိုင်း Agent အရင်ဖတ်တဲ့ Project စည်းကမ်းဖိုင်
Skill	လိုတဲ့အခါမှ ဆွဲထုတ်ဖတ်တဲ့ လက်စွဲကျမ်း (SKILL.md)
Plan Mode	ကုန်မရေးခင် အစီအစဉ်ကို အရင် ချပြခိုင်းတဲ့ Mode
State	App ရဲ့ လက်ရှိ အခြေအနေ data (ဥပမာ — cart ထဲ ဘာတွေ ရှိလဲ)
Regression Test	တစ်ခါ ကိုက်ဖူးတဲ့ Bug ပြန်မပေါ်အောင် ကာကွယ်ထားတဲ့ Test
localStorage	Browser ထဲမှာ data သိမ်းတဲ့ နေရာ (ဝယ်သူ ဖုန်း/ကွန်ပျူတာထဲ)